arXiv:2512.21024v1 [cs.GT] 24 Dec 2025

POLICY-CONDITIONED POLICIES FOR MULTI-AGENT TASK SOLVING

Yue Lin[1], Shuhui Zhu[2], Wenhao Li[3], Ang Li[1], Dan Qiao[1],
Pascal Poupart[2,4], Hongyuan Zha[1], and Baoxiang Wang[1,4]

[1]The Chinese University of Hong Kong, Shenzhen
[2]University of Waterloo
[3]Tongji University
[4]Vector Institute
{yuelin,bxiangwang}@link.cuhk.edu.cn

ABSTRACT

In multi-agent tasks, the central challenge lies in the dynamic adaptation of strategies. However, directly conditioning on opponents' strategies is intractable in the prevalent deep reinforcement learning paradigm due to a fundamental "representational bottleneck": neural policies are opaque, high-dimensional parameter vectors that are incomprehensible to other agents. In this work, we propose a paradigm shift that bridges this gap by representing policies as human-interpretable source code and utilizing Large Language Models (LLMs) as approximate interpreters. This programmatic representation allows us to operationalize the game-theoretic concept of *Program Equilibrium*. We reformulate the learning problem by utilizing LLMs to perform optimization directly in the space of programmatic policies. The LLM functions as a point-wise best-response operator that iteratively synthesizes and refines the ego agent's policy code to respond to the opponent's strategy. We formalize this process as *Programmatic Iterated Best Response (PIBR)*, an algorithm where the policy code is optimized by textual gradients, using structured feedback derived from game utility and runtime unit tests. We demonstrate that this approach effectively solves several standard coordination matrix games and a cooperative Level-Based Foraging environment.

1 INTRODUCTION

The study of multi-agent tasks, typically formalized as Markov games (or stochastic games) (Shapley, 1953), presents a complexity that fundamentally transcends single-agent decision-making. In these settings, the environment is non-stationary from the perspective of any individual agent, as the state transitions and expected rewards are co-determined by the evolving policies of interacting agents. Consequently, there is no single optimal policy in isolation; rather, an agent must continuously compute a *best response* to the varying strategies of its opponents.

Canonical approaches to this problem, particularly in Multi-Agent Reinforcement Learning (MARL), typically rely on opponent modeling (He et al., 2016). Agents attempt to infer the current policies of their counterparts from the history of interactions. However, inferring complex, co-adapting policies from limited, high-dimensional, and non-stationary historical data remains an exceptionally difficult problem.

A theoretically more direct alternative is to explicitly condition an agent's policy on the *actual* policy of the opponent. In the domain of game theory, this ideal is captured by the concept of *Program Equilibrium* (Tennenholtz, 2004), where agents delegate decision-making to programs that can read and condition on each other's source code. This framework theoretically enables rich cooperative outcomes (e.g., via "mutual cooperation" proofs in the Prisoner's Dilemma) that are inaccessible to standard Nash equilibria.

However, Program Equilibrium has remained largely theoretical and disconnected from modern learning approaches. In the prevalent Deep Reinforcement Learning (DRL) paradigm, implementing such policy-conditioning is practically intractable due to a "representational bottleneck". Policies are represented as deep neural networks defined by millions of opaque parameters. Inputting one neural network's weights into another is computationally prohibitive and semantically ill-defined, as

arXiv:2512.21024v1 [cs.GT] 24 Dec 2025

用于多智能体任务求解的策略条件化策略

Yue Lin[1], Shuhui Zhu[2],Wenhao Li[3],Ang Li[1],Dan Qiao[1],
Pascal Poupart[2,4],Hongyuan Zha[1],和 Baoxiang Wang[1,4]

[1]香港中文大学（深圳） [2]滑铁卢大学[3]同济大学[4]
Vector Institute
{yuelin,bxiangwang}@link.cuhk.edu.cn

摘要

在多智能体任务中，核心挑战在于策略的动态适应。然而，在主流的深度强化学习范式中，由于存在根本性的“表示瓶颈”，直接以对手策略为条件难以实现：神经网络策略是不透明的高维参数向量，其他智能体无法理解。在本研究中，我们提出一种范式转变，通过将策略表示为人类可解释的源代码，并利用大语言模型（LLM）作为近似解释器来弥合这一鸿沟。这种程序化表示使我们能够将博弈论中的程序均衡概念付诸实践。我们利用大语言模型直接在程序化策略空间中进行优化，从而重新构建学习问题。大语言模型充当逐点最优响应算子，通过迭代合成和改进自我智能体的策略代码来响应对手策略。我们将这一过程形式化为程序化迭代最优响应（*PIBR*）算法，该算法利用从博弈效用和运行时单元测试中获得的结构化反馈，通过文本梯度优化策略代码。我们证明，该方法能够有效求解若干标准协调矩阵博弈，以及一个合作式基于等级的觅食环境。

1 引言

多智能体任务通常被形式化为马尔可夫博弈（或随机博弈）（Shapley，1953年），对这类任务的研究所涉及的复杂性从根本上超越了单智能体决策。在这些场景中，从任一智能体的视角来看，环境都是非平稳的，因为状态转移和期望奖励由相互交互的智能体不断演变的策略共同决定。因此，不存在孤立意义上的单一最优策略；相反，智能体必须针对对手不断变化的策略持续计算最优响应。

解决这一问题的经典方法，尤其是多智能体强化学习（MARL）中的方法，通常依赖于对手建模（He 等，2016年）。智能体试图根据交互历史推断对手当前的策略。然而，从有限、高维且非平稳的历史数据中推断复杂的协同适应策略，仍然是一个极其困难的问题。

一种在理论上更直接的替代方案是，明确地以对手的实际策略作为智能体策略的条件。在博弈论领域，这一理想体现为程序均衡（Tennenholtz，2004年）的概念：智能体将决策委托给程序，这些程序能够读取彼此的源代码，并以其为条件进行决策。该框架在理论上能够实现标准纳什均衡无法实现的丰富合作结果（例如，通过囚徒困境中的“相互合作”证明）。

然而，程序均衡在很大程度上仍停留在理论层面，与现代学习方法脱节。在当前主流的深度强化学习（DRL）范式中，由于存在“表征瓶颈”，实现这种策略条件化在实践中几乎不可行。策略被表示为由数百万个不透明参数定义的深度神经网络。将一个神经网络的权重输入另一个神经网络，不仅计算成本高得难以承受，而且在语义上也缺乏明确定义，因为

the parameter vector is a high-dimensional, unstructured, and non-unique representation of behavior (Goodfellow et al., 2016).

The central thesis of this work is that we can overcome this bottleneck by leveraging Large Language Models (LLMs). We propose representing policies as *programmatic policies*, which are executable, human-interpretable source code. Unlike opaque tensor parameters, source code is semantically dense and structured. By utilizing LLMs as approximate interpreters, we can operationalize the game-theoretic concept of Program Equilibrium within a learning-based framework. This allows agents to condition on the opponent's source code, enabling rich cooperative outcomes that are theoretically inaccessible to standard Nash equilibria but have previously been difficult to realize in practice.

This paradigm shift enables us to operationalize policy-conditional adaptation in a fundamentally new way. Instead of searching for a static policy via gradient descent on parameters, we shift the optimization landscape to the code space, where the LLM functions as a point-wise best-response operator. Given the source code of an opponent's policy, the LLM generates and iteratively refines the source code of the best-response policy.

We formalize this learning process as *Programmatic Iterated Best Response (PIBR)*. This algorithm proceeds as an iterative dynamic, where at each step, an agent's policy code is optimized against the fixed code of its opponents. The optimization is driven by modern textual gradient techniques (Yuksekgonul et al., 2024) using structured feedback, incorporating both (1) utility feedback from game trajectories and (2) unit test feedback to ensure syntactic and runtime correctness. By doing so, we present a practical instantiation of Program Equilibrium for general Markov games, demonstrating efficacy in several standard coordination matrix games and a cooperative grid-world task named `Level-Based Foraging`.

1.1 Relation to Concurrent Work

During the preparation of this manuscript, a contemporaneous work titled "Evaluating LLMs in Open-Source Games" (Sistla & Kleiman-Weiner, 2025a) was presented at NeurIPS 2025. The work introduced a framework for Open-Source Games, evaluating how LLMs perform in game-theoretic settings when agents have full visibility of each other's source code. We acknowledge that both the work and our work share a fundamental contribution: identifying that LLMs can serve as approximate interpreters to bridge the gap between theoretical Program Equilibrium and practical implementation. Both approaches utilize the transparency of code to enable conditional cooperation that is difficult to achieve with black-box neural policies. Chronologically, Sistla & Kleiman-Weiner (2025a) are the first to make this contribution.

However, our work diverges significantly from Sistla & Kleiman-Weiner (2025a) in motivation, methodology, and experimental scope.

1. **Motivation from MARL Challenges:** While Sistla & Kleiman-Weiner (2025a) focuses on evaluating LLMs within a game-theoretic context, our approach stems fundamentally from the perspective of MARL. We identify programmatic policies specifically as a solution to the "representational bottleneck" and non-stationarity in MARL. We frame this not merely as an evaluation of LLMs, but as a necessary paradigm shift to solve the recursive reasoning challenges inherent in learning co-adapting policies.
2. **Optimization via Textual Gradients:** Sistla & Kleiman-Weiner (2025a) primarily relies on the inherent reasoning capabilities of LLMs via prompting. In contrast, we propose *Programmatic Iterated Best Response (PIBR)* as a formal optimization algorithm. Our method explicitly treats the LLM as a definable operator optimized via textual gradients (Yuksekgonul et al., 2024). We incorporate a structured feedback loop that combines game utility with runtime unit tests, ensuring that the generated policies are not only strategically sound but also syntactically robust and executable.
3. **Complexity of Tasks:** We extend the validation of programmatic policies beyond standard matrix games (which are the primary focus of concurrent works) to the *Level-Based Foraging* environment. This is a complex grid-world task requiring spatial coordination and sequential decision-making, demonstrating the capability of our method to handle higher-dimensional state spaces and more intricate coordination dynamics.

参数向量是行为的一种高维、无结构且非唯一的表征（Goodfellow 等人，2016年）。

本研究的核心论点是，我们可以利用大语言模型（LLM）突破这一瓶颈。我们提出将策略表示为程序化策略，即可执行且人类可理解的源代码。与不透明的张量参数不同，源代码具有丰富的语义和清晰的结构。通过将大语言模型用作近似解释器，我们可以在基于学习的框架中将博弈论中的程序均衡概念付诸实现。这使智能体能够以对手的源代码为条件进行决策，从而实现丰富的合作结果；这些结果在理论上是标准纳什均衡无法达到的，但此前也难以在实践中实现。

这一范式转变使我们能够以一种全新的方式实现策略条件适应。我们不再通过对参数进行梯度下降来搜索静态策略，而是将优化景观转移到代码空间，在其中由大语言模型充当逐点最优响应算子。给定对手策略的源代码，大语言模型会生成最优响应策略的源代码，并对其进行迭代改进。

我们将这一学习过程形式化为程序化迭代最优响应（*PIBR*）。该算法以迭代动态过程运行，在每一步中，针对对手的固定代码优化智能体的策略代码。优化由现代文本梯度技术（Yuksekgonul 等人，2024年）驱动，并使用结构化反馈，其中包括（1）来自博弈轨迹的效用反馈，以及（2）用于确保语法与运行时正确性的单元测试反馈。通过这种方式，我们为一般马尔可夫博弈中的程序均衡提供了一种实用的具体实现，并在若干标准协调矩阵博弈以及名为基于等级的觅食的合作式网格世界任务中展示了其有效性。

1.1 与同期工作的关系

在本文撰写期间，一项题为“在开源博弈中评估大语言模型”的同期工作（Sistla & Kleiman-Weiner，2025a）在 NeurIPS 2025 上发表。该工作提出了一个开源博弈框架，用于评估当智能体能够完全查看彼此的源代码时，大语言模型在博弈论情境中的表现。我们认可，该工作与我们的工作具有一项共同的基础性贡献：指出大语言模型可以充当近似解释器，弥合理论上的程序均衡与实际实现之间的鸿沟。两种方法都利用代码的透明性，实现黑箱神经网络策略难以达成的条件合作。从时间顺序来看，Sistla & Kleiman-Weiner（2025a）最先作出了这一贡献。

然而，我们的工作与 Sistla & Kleiman-Weiner （2025a）在研究动机、方法和实验范围上存在显著差异。

1. **源于多智能体强化学习挑战的研究动机：**尽管 Sistla &Kleiman-Weiner （2025a）侧重于在博弈论背景下评估大语言模型，但我们的方法从根本上源于多智能体强化学习的视角。我们明确将程序化策略视为解决多智能体强化学习中“表征瓶颈”和非平稳性问题的方案。我们认为，这不仅仅是对大语言模型的评估，更是解决协同适应策略学习中固有的递归推理挑战所必需的范式转变。
2. **通过文本梯度进行优化：**Sistla &Kleiman-Weiner （2025a）主要通过提示来利用大语言模型固有的推理能力。相比之下，我们提出程序化迭代最优响应（*PIBR*）作为一种形式化的优化算法。我们的方法明确将大语言模型视为一个可定义的算子，并通过文本梯度对其进行优化（Yuksekgonul 等，2024年）。我们引入了一个结合博弈效用与运行时单元测试的结构化反馈循环，确保生成的策略不仅在策略层面合理，而且在语法层面稳健且可执行。

3. **任务复杂性：**我们将程序化策略的验证范围从标准矩阵博弈（同期工作的主要关注点）扩展至基于等级的觅食环境。这是一项复杂的网格世界任务，需要空间协调和序贯决策，展示了我们的方法处理更高维状态空间和更复杂协调动态的能力。

Acknowledging the precedence of Sistla & Kleiman-Weiner (2025a), we refrain from submitting this specific version for formal conference publication to adhere to novelty standards. Instead, we release this manuscript as a technical report to establish the timeline of our independent contribution. We plan to incorporate these findings into future submissions that feature significant technical improvements or novel extensions beyond the scope of the current discussion.

2 PRELIMINARIES

This section provides the necessary background for our method. We begin by establishing the formalisms for multi-agent learning, including game settings and solution concepts. Then we discuss the two fundamental challenges that motivate our work: non-stationarity and the complexities of opponent modeling. Finally, we introduce the concept of program equilibrium and provability logic, which provide the theoretical basis for agents that can condition their policies on the source code of their opponents. A detailed discussion of related work is deferred to Appendix A.

2.1 THE GAME

In this work, we focus on the multi-agent task which is formulated as a *Markov Game* (Littman, 1994), also known as a stochastic game (Shapley, 1953). A Markov Game for N agents is defined by a tuple $\mathcal{G} = \langle N, \mathbb{S}, \{\mathbb{A}^i\}_{i=1}^N, \mathcal{T}, \{r^i\}_{i=1}^N, \gamma \rangle$. $\mathbb{S}$ is the set of global states. $\mathbb{A}^i$ is the action space for agent i, and the joint action space is $\mathbb{A} = \times_{i=1}^N \mathbb{A}^i$. The function $\mathcal{T} : \mathbb{S} \times \mathbb{A} \to \Delta(\mathbb{S})$ is the state transition probability function. Each agent i has a reward function $r^i : \mathbb{S} \times \mathbb{A} \to \mathbb{R}$, and $\gamma \in [0, 1)$ is the discount factor. At each timestep t, agents simultaneously select actions $a_t^i \in \mathbb{A}^i$. The system transitions to $s_{t+1} \sim \mathcal{T}(\cdot \mid s_t, \boldsymbol{a}_t)$ and each agent i receives a reward $r_t^i = r^i(s_t, \boldsymbol{a}_t)$, where $\boldsymbol{a}_t$ is the joint action.

2.2 THE SOLUTION CONCEPTS

The objective of each agent i is to find an optimal policy π^{i*} that, when combined with the policies of all other agents, maximizes its *expected discounted return*. This cumulative reward, denoted J^i, is defined for a joint policy $\boldsymbol{\pi} = (\pi^1, \ldots, \pi^N)$ as:

$$J^i(\boldsymbol{\pi}) = \mathbb{E}_{\langle \mathcal{G}, \boldsymbol{\pi} \rangle} \left[\sum_{t=0}^{\infty} \gamma^t r^i(s_t, \boldsymbol{a}_t) \right],$$

where $\mathbb{E}_{\langle \mathcal{G}, \boldsymbol{\pi} \rangle}[\cdot]$ denotes the expectation over the state transitions $\mathcal{T}$ and the agents' joint actions $\boldsymbol{\pi}$.

The primary solution concept is the *Nash Equilibrium (NE)* (Nash Jr, 1950), a joint policy $\boldsymbol{\pi}^* = (\pi^{i*}, \boldsymbol{\pi}^{-i*})$ from which no single agent i has an incentive to unilaterally deviate. Formally, $\boldsymbol{\pi}^*$ is a Nash Equilibrium if for all $i \in N$ and for all alternative policies π^i available to agent i ($\pi^i \in \Pi^i$), the following condition holds:

$$J^i(\boldsymbol{\pi}^*) \geq J^i(\pi^i, \boldsymbol{\pi}^{-i*}).$$

This general definition is refined based on the game's temporal horizon and the class of policies considered.

In the finite-horizon (episodic) setting, equilibria must hold at all decision points. The history h_t is defined as $h_t = (s_0, \boldsymbol{a}_0, \ldots, s_{t-1}, \boldsymbol{a}_{t-1}, s_t)$, and $\mathbb{H}$ is the space of all such histories. One refinement is the *Subgame Perfect Equilibrium (SPE)* (Bielefeld, 1988), which applies when players use history-dependent policies, $\pi^i : \mathbb{H} \to \Delta(\mathbb{A}^i)$, denoted $\Pi^i_{\mathbb{H}}$. An SPE $\boldsymbol{\pi}^*$ requires the NE condition to hold in every subgame, i.e., beginning from any possible history h_t. For all $i \in N$, $h_t \in \mathbb{H}$, and $\pi^i \in \Pi^i_{\mathbb{H}}$:

$$J^i(\boldsymbol{\pi}^* \mid h_t) \geq J^i((\pi^i, \boldsymbol{\pi}^{-i*}) \mid h_t),$$

where $J^i(\boldsymbol{\pi} \mid h_t) = \mathbb{E}_{\langle \mathcal{G}, \boldsymbol{\pi} \rangle} \left[\sum_{k=t}^{\infty} \gamma^{k-t} r^i(s_k, \boldsymbol{a}_k) \mid h_t \right]$ is the expected future discounted reward given history h_t. A second refinement is the *Markov Perfect Equilibrium (MPE)* (Fudenberg & Tirole, 1991), which restricts policies to be Markovian and time-dependent, $\pi^i : \mathbb{S} \times \mathbb{T} \to \Delta(\mathbb{A}^i)$, denoted $\Pi^i_{\mathbb{M}}$. An MPE $\boldsymbol{\pi}^*$ requires the NE condition to hold at every state-time pair (s, t). For all $i \in N$, $s \in \mathbb{S}$, $t \in \mathbb{T}$, and $\pi^i \in \Pi^i_{\mathbb{M}}$:

$$J^i(\boldsymbol{\pi}^* \mid s, t) \geq J^i((\pi^i, \boldsymbol{\pi}^{-i*}) \mid s, t),$$

鉴于Sistla& Kleiman-Weiner（2025a）的工作先于我们，为遵守新颖性标准，我们不将此特定版本提交至会议正式发表。相反，我们将此稿件作为技术报告发布，以明确我们独立贡献的时间线。我们计划将这些发现纳入未来的投稿中，届时将包含超出当前讨论范围的重大技术改进或创新性扩展。

2 预备知识

本节介绍我们的方法所需的背景知识。首先，我们建立多智能体学习的形式化框架，包括博弈设定和解概念。随后，我们讨论推动本研究的两个根本性挑战：非平稳性和对手建模的复杂性。最后，我们介绍程序均衡和可证性逻辑的概念，它们为智能体根据对手的源代码制定自身策略提供了理论基础。相关工作的详细讨论见附录 A。

2.1 博弈

在本研究中，我们关注被形式化为马尔可夫博弈（Littman，1994年）的多智能体任务，马尔可夫博弈也称为随机博弈（夏普利，1953年）。一个包含 N 个智能体的马尔可夫博弈由元组 $\mathcal{G} = \langle N$、$\mathbb{S}$、$\{\mathbb{A}^i\}_{i=1}^N$、$\mathcal{T}$、$\{r^i\}_{i=1}^N$、$\gamma \rangle$ 定义。$\mathbb{S}$ 是全局状态的集合。$\mathbb{A}^i$ 是智能体 i 的动作空间，联合动作空间为 $\mathbb{A} = \times_{i=1}^N \mathbb{A}^i$。函数 $\mathcal{T}$：$\mathbb{S} \times \mathbb{A} \to \Delta(\mathbb{S})$ 是状态转移概率函数。每个智能体 i 都有一个奖励函数 r^i：$\mathbb{S} \times \mathbb{A} \to \mathbb{R}$，而 $\gamma \in [0, 1)$ 是折扣因子。在每个时间步 t，智能体同时选择动作 $a_t^i \in \mathbb{A}^i$。系统转移至 $s_{t+1} \sim \mathcal{T}(\cdot \mid s_t, \boldsymbol{a}_t)$，每个智能体 i 获得奖励 $r_t^i = r^i(s_t, \boldsymbol{a}_t)$，其中 $\boldsymbol{a}_t$ 是联合动作。

2.2 解概念

每个智能体 i 的目标是找到一个最优策略 π^{i*}，使其与所有其他智能体的策略相结合时，能够最大化自身的期望折扣回报。这一累积奖励记为 J^i，对于联合策略 $\boldsymbol{\pi} = (\pi^1, \ldots, \pi^N)$，其定义为：

$$J^i(\boldsymbol{\pi}) = \mathbb{E}_{\langle \mathcal{G}, \boldsymbol{\pi} \rangle} \left[\sum_{t=0}^{\infty} \gamma^t r^i(s_t, \boldsymbol{a}_t) \right],$$

whe其中，$\mathbb{E}_{\langle \mathcal{G}, \boldsymbol{\pi} \rangle}[\cdot]$ 表示对状态转移 $\mathcal{T}$和智能体的联合动作 $\boldsymbol{\pi}$求期望 .

主要的解概念是纳什均衡（*NE*）（小纳什，1950年），即一种联合策略 $\boldsymbol{\pi}^* = (\pi^{i*}, \boldsymbol{\pi}^{-i*})$，在该策略下，任何单个智能体 i 都没有单方面偏离的动机。形式上，$\boldsymbol{\pi}^*$ 是纳什均衡，当且仅当对于所有 $i \in N$ 以及所有替代策略 π^i（可供智能体 i（$\pi^i \in \Pi^i$）采用），以下条件均成立：

$$J^i(\boldsymbol{\pi}^*) \geq J^i(\pi^i, \boldsymbol{\pi}^{-i*}).$$

Thi这一一般定义根据博弈的时间范围和策略类别加以精炼 s
con所考虑的。

在有限时域（回合式）设定中，均衡必须在所有决策点上成立。历史 h_t 定义为 $h_t = (s_0, \boldsymbol{a}_0, \ldots, s_{t-1}, \boldsymbol{a}_{t-1}, s_t)$，而 $\mathbb{H}$ 是所有此类历史构成的空间。一种精炼概念是子博弈精炼均衡（SPE）（Bielefeld，1988年），适用于参与者采用依赖历史的策略 π^i：$\mathbb{H} \to \Delta(\mathbb{A}^i)$的情形，这类策略记为 $\Pi^i_{\mathbb{H}}$。子博弈精炼均衡 $\boldsymbol{\pi}^*$ 要求纳什均衡条件在每个子博弈中均成立，即从任意可能的历史 h_t开始均成立。对于所有 $i \in N$、$h_t \in \mathbb{H}$和 $\pi^i \in \Pi^i_{\mathbb{H}}$：

其中，$J^i(\boldsymbol{\pi} \mid h_t) = \mathbb{E}_{\langle \mathcal{G}, \boldsymbol{\pi} \rangle} \left[\sum_{k=t}^{\infty} \gamma^{k-t} r^i(s_k, \boldsymbol{a}_k) \mid h_t \right]$ 是在给定历史 h_t 下的预期未来折扣回报。第二种精炼是马尔可夫精炼均衡（MPE）（Fudenberg & Tirole，1991年），它将策略限制为具有马尔可夫性且依赖于时间的策略，π^i：$\mathbb{S} \times \mathbb{T} \to \Delta(\mathbb{A}^i)$，记为 $\Pi^i_{\mathbb{M}}$。马尔可夫精炼均衡 $\boldsymbol{\pi}^*$ 要求纳什均衡条件在每个状态-时间对 (s, t) 上均成立。对于所有$i \in N$、$s \in \mathbb{S}$、$t \in \mathbb{T}$ 和 $\pi^i \in \Pi^i_{\mathbb{M}}$：

$$J^i(\boldsymbol{\pi}^* \mid s, t) \geq J^i((\pi^i, \boldsymbol{\pi}^{-i*}) \mid s, t),$$

where $J^i(\boldsymbol{\pi} \mid s,t) = \mathbb{E}_{\langle \mathcal{G},\boldsymbol{\pi}\rangle}\left[\sum_{k=t}^{\infty}\gamma^{k-t}r^i(s_k,\boldsymbol{a}_k) \mid s_t = s\right]$ is the expected future discounted reward from state s at time t. Both SPE and finite-horizon MPE can be computed via backward induction (Farina, 2024).

In the infinite-horizon discounted setting, a foundational result establishes that a Nash Equilibrium exists in *stationary, Markovian policies*, $\pi^i : \mathbb{S} \to \Delta(\mathbb{A}^i)$ (Fink, 1964; Takahashi, 1964). Formally, a joint stationary policy $\boldsymbol{\pi}^* = (\pi^{1*}, \ldots, \pi^{N*})$ is an equilibrium if for all $i \in N$ and for all alternative policies $\pi^{i\prime} \in \Pi^i_{\text{all}}$ (where Π^i_{all} includes arbitrary history-dependent policies):

$$J^i(\boldsymbol{\pi}^*) \geq J^i(\pi^{i\prime}, \boldsymbol{\pi}^{-i*}).$$

This demonstrates that the stationary equilibrium policy π^{i*} is robust against deviations to any non-Markovian strategy.

2.3 Learning Methods for Markov Games

The advancement of DRL has enabled the use of deep neural networks to approximate complex policies and value functions, greatly extending the applicability of RL to high-dimensional state and action spaces. MARL extends this DRL framework to environments with multiple interacting agents ($N > 1$). These agents may be cooperative, competitive, or operate in a mixed setting.

In a MARL setting, each agent i learns its own *policy* π^i. As defined in the previous section, this policy can be a mapping from the current state (a stationary, Markovian policy $\pi^i : \mathbb{S} \to \Delta(\mathbb{A}^i)$) or from the full history (a history-dependent policy $\pi^i : \mathbb{H} \to \Delta(\mathbb{A}^i)$). In practice, policies are frequently history-dependent. Agents require history for two principal reasons: **(1)** First, in partially observable environments, an agent must utilize its history to infer the current underlying state. We assume a fully observable environment, so this point does not apply. **(2)** Second, history is employed to estimate the policies of other agents. Notably, even in fully observable environments where the state is known, history remains essential for decision-making if the policies of other players are not directly accessible. This necessity for opponent modeling will be further discussed in Section 2.5.

The *policy learning rule* dictates how the agent updates its policy parameters, θ^i (when π^i is a parameterized function $\pi^i_{\theta^i}$), to maximize this objective. The general learning rule involves an iterative update mechanism, typically derived from gradient-based optimization:

$$\theta^i_{k+1} \leftarrow \theta^i_k + \alpha \nabla_{\theta^i} J^i(\boldsymbol{\pi}),$$

where k is the iteration index and α is the learning rate. We can abstract this update step for agent i as a *learning dynamics* $\mathcal{L}^i$, such that:

$$\theta^i_{k+1} = \mathcal{L}^i(\theta_k).$$

This is a *vanilla* learning dynamics in an independent learning style, which considers its own parameters only. The gradient $\nabla_{\theta^i} J^i(\boldsymbol{\pi})$ is estimated using various techniques such as policy gradient methods (e.g., REINFORCE (Williams, 1992; Sutton et al., 1999), Actor-Critic (Konda & Tsitsiklis, 1999)) or Q-learning variants, where the form of the gradient estimate strongly depends on the specific MARL algorithm employed.

2.4 Non-Stationarity

The fundamental challenge that distinguishes multi-agent learning from single-agent learning is **non-stationarity** (Hernandez-Leal et al., 2017). In a single-agent system, the environment is governed by a stationary MDP. In contrast, in a multi-agent system, the effective environmental dynamics from the perspective of an ego agent i are co-determined by the changing policies $\boldsymbol{\pi}^{-i} = (\pi^j)_{j\neq i}$ of all other agents. We can denote the subjective view of the environment and opponents from agent i as the *perspective environment* $\mathcal{G}^i$:

$$\mathcal{G}^i = \langle \mathcal{G}, \boldsymbol{\pi}, \{\mathcal{L}^j\}_{j\in N}\rangle,$$

and this perspective environment is non-stationary, i.e., even if the objective environment $\mathcal{G}$ and the ego agent's policy remain static, the ego agent's expected discounted return J^i may change, due to the updates of other agents' policies.

Specifically, consider the vanilla learning rule as described above (i.e., the independent learning style), when any opponent $j \neq i$ updates its policy parameters via its learning rule $\mathcal{L}^j$ (i.e., $\theta^j_{k+1} =$

其中，$J^i(\boldsymbol{\pi} \mid s,t) = \mathbb{E}_{\langle \mathcal{G},\boldsymbol{\pi}\rangle}\left[\sum_{k=t}^{\infty}\gamma^{k-t}r^i(s_k,\boldsymbol{a}_k) \mid s_t = s\right]$ 是从状态 s 在时间 t开始的预期未来折扣回报。子博弈精炼均衡和有限时域马尔可夫完美均衡均可通过逆向归纳法计算（Farina，2024年）。

在无限时域折扣设定下，一项基础性结果表明，纳什均衡存在于平稳马尔可夫策略中，$\pi^i : \mathbb{S} \to \Delta(\mathbb{A}^i)$（Fink，1964年；Takahashi，1964年）。形式上，联合平稳策略 $\boldsymbol{\pi}^* = (\pi^{1*}, \ldots, \pi^{N*})$ 是均衡，当且仅当对于所有 $i \in N$ 以及所有替代策略 $\pi^{i\prime} \in \Pi^i_{\text{all}}$（其中 Π^i_{all} 包括任意历史依赖策略），均有：

$$J^i(\boldsymbol{\pi}^*) \geq J^i(\pi^{i\prime}, \boldsymbol{\pi}^{-i*}).$$

这表明，平稳均衡策略 π^{i*} 对于偏离至任何非马尔可夫策略的行为都具有稳健性。

2.3 马尔可夫博弈的学习方法

深度强化学习的发展使得利用深度神经网络近似复杂策略和价值函数成为可能，大大拓展了强化学习在高维状态与动作空间中的适用范围。多智能体强化学习将这一深度强化学习框架扩展至包含多个相互作用的智能体的环境（$N > 1$）。这些智能体可能相互合作、相互竞争，也可能在混合情境下运行。

在多智能体强化学习场景中，每个智能体 i 都学习自己的 策略 π^i。如上一节所定义，该策略可以是从当前状态出发的映射（平稳的马尔可夫策略 $\pi^i : \mathbb{S} \to \Delta(\mathbb{A}^i)$），也可以是从完整历史出发的映射（历史依赖策略 $\pi^i : \mathbb{H} \to \Delta(\mathbb{A}^i)$）。在实践中，策略往往依赖于历史。智能体需要历史主要有两个原因：**(1)**首先，在部分可观测环境中，智能体必须利用其历史来推断当前的底层状态。我们假设环境为完全可观测环境，因此这一点不适用。**(2)** 其次，历史被用于估计其他智能体的策略。值得注意的是，即使在状态已知的完全可观测环境中，如果无法直接获取其他参与者的策略，历史对于决策仍然至关重要。对手建模的这一必要性将在第 2.5 节中进一步讨论。

策略学习规则规定了智能体如何更新其策略参数 θ^i （当 π^i 是参数化函数 $\pi^i_{\theta^i}$时），以最大化这一目标。一般的学习规则涉及一种迭代更新机制，通常由基于梯度的优化推导而来：

$$\theta^i_{k+1} \leftarrow \theta^i_k + \alpha \nabla_{\theta^i} J^i(\boldsymbol{\pi}),$$

whe其中，k 为迭代索引，α 为学习率。我们可以将智能体的这一更新步骤抽象为 i as a 学习动力学 $\mathcal{L}^i$，使得：

$$\theta^i_{k+1} = \mathcal{L}^i(\theta_k).$$

这是一种基础的独立学习式学习动力学，仅考虑自身的参数。梯度 $\nabla_{\theta^i} J^i(\boldsymbol{\pi})$ 可通过多种技术进行估计，例如策略梯度方法（如 REINFORCE（Williams，1992年；Sutton 等人，1999年）、演员-评论家（Konda 与 Tsitsiklis，1999年））或 Q学习的变体，其中梯度估计的形式在很大程度上取决于所采用的具体多智能体强化学习算法。

2.4 非平稳性

多智能体学习区别于单智能体学习的根本挑战是**非平稳性**（Hernandez-Leal 等人，2017年）。在单智能体系统中，环境由平稳马尔可夫决策过程支配。相比之下，在多智能体系统中，从自我智能体 i 的视角来看，实际的环境动力学由所有其他智能体不断变化的策略 $\boldsymbol{\pi}^{-i} = (\pi^j)_{j\neq i}$共同决定。我们可以将智能体$i$ 对环境和对手的主观视角记为视角环境 $\mathcal{G}^i$：

$$\mathcal{G}^i = \langle \mathcal{G}, \boldsymbol{\pi}, \{\mathcal{L}^j\}_{j\in N}\rangle,$$

而且，这种视角环境是非平稳的，即，即使客观环境 $\mathcal{G}$ 和自我智能体的策略保持不变，自我智能体的期望折扣回报 J^i 也可能因其他智能体的策略更新而发生变化。

具体而言，考虑上述基础学习规则（即独立学习方式），当任意对手 $j \neq i$ 通过其学习规则 $\mathcal{L}^j$ 更新自身的策略参数时（即 $\theta^j_{k+1} =$

$\mathcal{L}^j\left(\theta_k^j\right)$, this results in a change to its policy $\pi_k^j \to \pi_{k+1}^j$ (where π_k^j is shorthand for $\pi_{\theta_k^j}^j$). This, in turn, changes the joint policy of all opponents, $\boldsymbol{\pi}_k^{-i} \to \boldsymbol{\pi}_{k+1}^{-i}$. Consequently, even if agent i's policy π^i remains static, its expected discounted return J^i is no longer stationary:

$$J^i(\pi^i, \boldsymbol{\pi}_k^{-i}) \neq J^i(\pi^i, \boldsymbol{\pi}_{k+1}^{-i}).$$

This instability invalidates the convergence guarantees of standard single-agent RL algorithms.

2.5 Opponent Modeling

To succeed in this non-stationary landscape, an agent i must adapt its policy to the changing policies of its opponents. The goal shifts from finding a single optimal policy to dynamically computing a *best response (BR)* to the current policies of the other agents.

Calculating a BR presupposes that the agent possesses an estimate of its opponents' policies, $\hat{\boldsymbol{\pi}}^{-i}$. This inevitably leads to the need for *opponent modeling* (He et al., 2016): the process of defining a function m that maps available data (e.g., interaction history) to an estimate of the opponent policies:

$$\hat{\boldsymbol{\pi}}^{-i} = m(h_t).$$

To best respond to a hypothesized opponent policy, $\hat{\pi}^j$, the opponent modeling component $m(h_t)$ serves as a direct input to the agent's response policy. For example, a history-dependent best-response policy is defined as $\pi_{\text{br}}^i : \Pi^{-i} \times \mathbb{H} \to \Pi^i$. With this policy, player i takes an action a_t^i at timestep t as:

$$a_t^i \sim \pi_{\text{br}}^i(\cdot \mid \hat{\boldsymbol{\pi}}^{-i}, h_t) = \pi_{\text{br}}^i(\cdot \mid m(h_t), h_t).$$

It is important to note that inferring an opponent's policy from the interaction history h_t presents two primary challenges. **(1)** First, the challenge of ***policy drift***: all policies are constantly updating, meaning the interaction data is not sampled from a stable joint policy distribution. **(2)** Second, all agents are simultaneously trying to model each other, which introduces a complex hierarchy of beliefs (Gmytrasiewicz & Doshi, 2005; Rabinowitz et al., 2018). For example, agent i must not only model what agent j will do, but also model what agent j guesses agent i will do, and so on, leading to an ***arbitrary-level recursive reasoning challenge*** (Gmytrasiewicz & Doshi, 2005).

2.6 Program Equilibrium and Provability Logic

In canonical game theory, players select actions or strategies directly. The concept of *Program Equilibrium* (Tennenholtz, 2004) departs from this by assuming that players delegate decision-making to computer programs. Crucially, these programs are capable of reading and conditioning their execution on the *source code* of their opponent's program.

Formally, let $\mathbb{A}^i$ denote the action space for player i. Let Π_{prog} denote the set of all valid computer programs in a given Turing-complete language. A policy for player i is a program $\pi^i \in \Pi_{\text{prog}}$. To be rigorous, we explicitly distinguish between the program's execution logic (semantics) and its representation (syntax):

- **Semantics (π^i):** We identify the policy π^i directly with the computable function it implements upon execution.
- **Syntax ($\ulcorner \pi^i \urcorner$):** The Gödel number or string representation of the program, where $\ulcorner \cdot \urcorner : \Pi_{\text{prog}} \to \{0,1\}^*$ is the encoding mapping.

Since π^i acts as a function that takes the opponent's source code as input and maps it to a probability distribution over actions, the interaction is formally defined by:

$$\pi^i : \{0,1\}^* \to \Delta(\mathbb{A}^i)$$

Specifically, in a game, player i executes π^i on input $\ulcorner \boldsymbol{\pi}^{-i} \urcorner$ to determine their action $a^i \sim \pi^i(\ulcorner \boldsymbol{\pi}^{-i} \urcorner)$. This meta-game structure enables outcomes that are inaccessible in standard Nash equilibria. For example, in the one-shot Prisoner's Dilemma, mutual cooperation becomes sustainable if players submit programs that verify whether the opponent's source code is functionally equivalent to their own (or satisfies a specific cooperative predicate) before cooperating.

$\mathcal{L}^j\left(\theta_k^j\right)$，这会导致其策略 $\pi_k^j \to \pi_{k+1}^j$ 发生变化（其中， π_k^j 是 $\pi_{\theta_k^j}^j$ 的简写）。这进而改变了所有对手的联合策略 $\boldsymbol{\pi}_k^{-i} \to \boldsymbol{\pi}_{k+1}^{-i}$。因此，即使智能体 i的策略π^i 保持不变，其期望折扣回报 J^i 也不再平稳：

$$J^i(\pi^i, \boldsymbol{\pi}_k^{-i}) \neq J^i(\pi^i, \boldsymbol{\pi}_{k+1}^{-i}).$$

这种不稳定性使标准单智能体强化学习算法的收敛保证失效。

2.5 对手建模

要在这种非平稳环境中取得成功，智能体 i 必须调整自身策略，以适应对手不断变化的策略。目标从寻找单一的最优策略转变为针对其他智能体的当前策略动态计算最佳响应（*BR*）。

计算最优响应的前提是，智能体拥有对手策略的估计值 $\hat{\boldsymbol{\pi}}^{-i}$。这必然引出了对对手建模（He 等，2016年）的需求：即定义一个函数 m，将可用数据（例如交互历史）映射为对手策略的估计值：

$$\hat{\boldsymbol{\pi}}^{-i} = m(h_t).$$

为了对假设的对手策略 $\hat{\pi}^j$作出最优响应，对手建模组件 $m(h_t)$ 直接作为智能体响应策略的输入。例如，依赖历史的最优响应策略定义为 π_{br}^i： $\Pi^{-i} \times \mathbb{H} \to \Pi^i$。采用该策略时，参与者 i 在时间步 t采取动作 a_t^i的方式如下：

$$a_t^i \sim \pi_{\text{br}}^i(\cdot \mid \hat{\boldsymbol{\pi}}^{-i}, h_t) = \pi_{\text{br}}^i(\cdot \mid m(h_t), h_t).$$

需要注意的是，根据交互历史推断对手的策略 h_t 面临两大主要挑战。（**1**）首先是策略漂移的挑战：所有策略都在不断更新，这意味着交互数据并非采样自稳定的联合策略分布。（**2**）其次，所有智能体都在同时尝试对彼此建模，这引入了复杂的信念层级（Gmytrasiewicz 与 Doshi，2005年；Rabinowitz 等人，2018年）。例如，智能体 i 不仅必须对智能体 j 将采取的动作建模，还必须对智能体 j 如何猜测智能体 i 将采取的动作建模，如此递推，从而带来任意层级递归推理的挑战（Gmytrasiewicz 与 Doshi，2005年）。

2.6 程序均衡与可证明性逻辑学

在经典博弈论中，参与者直接选择动作或策略。程序均衡（Tennenholtz，2004年）这一概念则有所不同，它假设参与者将决策委托给计算机程序。关键在于，这些程序能够读取对手程序的源代码，并据此决定自身的执行方式。

形式上，令 $\mathbb{A}^i$ 表示参与者 i 的动作空间。令 Π_{prog} 表示给定图灵完备语言中所有有效计算机程序的集合。参与者 i 的策略是一个程序 $\pi^i \in \Pi_{\text{prog}}$。为确保严谨性，我们明确区分程序的执行逻辑（语义）与其表示（语法）：

- **语义（π^i）：** 我们将策略 π^i 直接等同于它在执行时实现的可计算函数。
- **语法（$\ulcorner \pi^i \urcorner$）：** 程序的哥德尔数或字符串表示，其中 $\ulcorner \cdot \urcorner : \Pi_{\text{prog}} \to \{0,1\}^*$ 为编码映射。

由于 π^i 作为一个函数，以对手的源代码为输入，并将其映射为动作上的概率分布，因此交互在形式上定义如下：

$$\pi^i : \{0,1\}^* \to \Delta(\mathbb{A}^i)$$

具体而言，在博弈中，参与者 i 以 $\ulcorner \boldsymbol{\pi}^{-i} \urcorner$ 为输入执行 π^i，以确定其动作 $a^i \sim \pi^i(\ulcorner \boldsymbol{\pi}^{-i} \urcorner)$。这种元博弈结构能够实现标准纳什均衡无法实现的结果。例如，在单次囚徒困境中，如果参与者提交的程序在合作之前会验证对手的源代码是否与自身的源代码在功能上等价（或是否满足某个特定的合作谓词），那么相互合作就能够维持。

A rigorous treatment of this self-referential structure, where program A analyzes program B analyzing program A, requires the machinery of *Provability Logic*. In this context, agents are modeled as searching for proofs within a formal system (e.g., Peano Arithmetic). Let $\Box P$ denote that the statement P is provable. A canonical example is `FairBot` (Barasz et al., 2014), an agent that cooperates if and only if it can prove that the opponent cooperates against it. Formally, let π_{FB} denote the `FairBot` program. It seeks a proof that the opponent's execution on its own source code yields cooperation:

$$\pi_{\text{FB}}(\ulcorner \boldsymbol{\pi}^{-i} \urcorner) = \text{“Cooperation”} \iff \Box(\boldsymbol{\pi}^{-i}(\ulcorner \pi_{\text{FB}} \urcorner) = \text{“Cooperation”})$$

While this creates a potential circular dependency, consistency is resolved via Löb's Theorem from mathematical logic, which states that for any formula P:

$$\Box(\Box P \to P) \to \Box P$$

This theorem implies that if an agent can prove "if my cooperation is provable, then it is true," it can indeed prove its own cooperation. This mechanism creates a *modal fixed point*, allowing mutual cooperation to be the provable outcome between two logical agents. This effectively establishes a ***Folk Theorem*** for program games, implying that any feasible and individually rational payoff profile can be sustained in a one-shot setting through conditional code commitment, achieving outcomes that are typically restricted to repeated interactions. An illustrative analysis of the program equilibrium within the Prisoner's Dilemma is provided in Appendix B.

Despite its theoretical power, canonical Program Equilibrium faces significant practical hurdles. Formal verification of arbitrary Turing-complete policies is computationally undecidable (by Rice's Theorem) and brittle to trivial syntactic variations. Our work can be viewed as a relaxation of this rigorous requirement. By utilizing LLMs as *approximate interpreters*, we operationalize a practical form of program equilibrium. Here, policies condition on the inferred *semantics* of the opponent's code rather than requiring formal deductive proofs, thereby bridging the gap between theoretical game-theoretic commitments and modern multi-agent learning.

3 METHOD

This section presents our methodology. We first articulate the "representational bottleneck" in deep MARL, which inhibits conditioning on opponent policies. We then introduce our core solution: a paradigm shift to programmatic policies. This shift enables us to lift optimization from policy space to operator space, where the objective becomes learning a point-wise best-response operator, φ^i, that maps opponent policy-code to ego-agent policy-code. Finally, we introduce the Programmatic Iterated Best Response (PIBR) algorithm, a learning framework designed to optimize this operator.

3.1 THE REPRESENTATIONAL BOTTLENECK OF DEEP REINFORCEMENT LEARNING

To address the two mentioned challenges in opponent modeling (the challenges of policy drift and arbitrary-level recursive reasoning), it is helpful to introduce the communication perspective. If an opponent j directly transmits its policy π^j via a signal $\sigma^j = \phi(\pi^j)$, the modeling task for agent i becomes:

$$\hat{\pi}^j = m(\sigma^j) = m(\phi(\pi^j)),$$

where ϕ is j's policy encoder and m is i's opponent modeling component which serves as a policy decoder. Notice that the opponent modeling for agents no longer needs to be inferred from historical data, but is instead communicated through signals between the agents.

Agent i takes an action sampled from its policy π^i_{br}, which is conditional on its estimation of opponents' policies:

$$a^i_t \sim \pi^i_{\text{br}}\left(\cdot \mid h_t, \hat{\pi}^j\right) = \pi^i_{\text{br}}\left(\cdot \mid h_t, m(\sigma^{-i})\right) = \pi^i_{\text{br}}\left(\cdot \mid h_t, m(\phi(\pi^j))\right).$$

The signal, conceptualized as a latent variable, must satisfy two critical and often competing constraints: **(1)** it must encapsulate and preserve maximal information pertaining to the policy of agent j; **(2)** it must be encoded in a representational format that is simultaneously tractable and parsable by the decoder or actor-network of agent i. Consequently, the selection of an appropriate signal space

要严格处理这种程序 A 分析程序 B、而后者又分析程序 A的自指结构，需要借助可证性逻辑的工具。在这一背景下，智能体被建模为在某个形式系统（例如皮亚诺算术）中搜索证明的实体。令 $\Box P$ 表示命题 P是可证明的。一个典型例子是 `FairBot` （Barasz 等人，2014年），这是一个当且仅当能够证明对手会与其合作时才会合作的智能体。形式化地，令 π_{FB} 表示 `FairBot`程序。它试图寻找一个证明，证明对手以该程序自身的源代码为输入执行时会选择合作：

$$\pi_{\text{FB}}(\ulcorner \boldsymbol{\pi}^{-i} \urcorner) = \text{“Cooperation”} \iff \Box(\boldsymbol{\pi}^{-i}(\ulcorner \pi_{\text{FB}} \urcorner) = \text{“Cooperation”})$$

尽管这会带来潜在的循环依赖，但可以通过数理逻辑中的勒布定理解决一致性问题。该定理指出，对于任意公式 P：

$$\Box(\Box P \to P) \to \Box P$$

该定理意味着，如果一个智能体能够证明“如果我的合作是可证明的，那么我确实会合作”，它就确实能够证明自身的合作。这一机制形成了一个模态不动点，使相互合作成为两个逻辑智能体之间可证明的结果。这实际上确立了程序博弈的一个无名氏定理，意味着任何可行的个体理性收益组合都可以通过条件性代码承诺在单次博弈情境中得到维持，从而实现通常只有通过重复交互才能实现的结果。附录 B 提供了囚徒困境中程序均衡的示例分析。

尽管经典程序均衡在理论上具有强大的能力，但它在实践中面临重大障碍。对任意图灵完备策略进行形式化验证在计算上是不可判定的（根据赖斯定理），而且即使是微小的语法变体也会使验证失效。我们的工作可以视为对这一严格要求的放宽。通过将大语言模型用作近似解释器，我们使一种实用形式的程序均衡得以实现。在这里，策略以推断出的对手代码的语义为条件，而不要求形式演绎证明，从而弥合了理论上的博弈论承诺与现代多智能体学习之间的鸿沟。

3 方法

本节介绍我们的方法。我们首先阐述深度多智能体强化学习中的“表征瓶颈”，这一瓶颈阻碍了以对手策略为条件进行建模。随后，我们介绍核心解决方案：向程序化策略的范式转变。这一转变使我们能够将优化从策略空间提升至算子空间，此时的目标变为学习一个逐点最优响应算子 φ^i，将对手策略代码映射为自身智能体策略代码。最后，我们介绍程序化迭代最优响应（PIBR）算法，这是一个旨在优化该算子的学习框架。

3.1 深度强化学习的表征瓶颈

为应对上述对手建模中的两项挑战（策略漂移和任意层级递归推理的挑战），引入通信视角会有所帮助。如果对手 j 通过信号 $\sigma^j = \phi$（π^j）直接传递其策略 π^j，则智能体 i的建模任务变为：

$$\hat{\pi}^j = m(\sigma^j) = m(\phi(\pi^j)),$$

其中，ϕ 是 j的策略编码器，m 是 i的对手建模组件，充当策略解码器。请注意，智能体的对手建模不再需要从历史数据中推断，而是通过智能体之间的信号进行通信来实现。

智能体 i 执行从其策略 π^i_{br} 中采样得到的动作，该策略以其对对手策略的估计为条件：

$$a^i_t \sim \pi^i_{\text{br}}\left(\cdot \mid h_t, \hat{\pi}^j\right) = \pi^i_{\text{br}}\left(\cdot \mid h_t, m(\sigma^{-i})\right) = \pi^i_{\text{br}}\left(\cdot \mid h_t, m(\phi(\pi^j))\right).$$

该信号被视为一个潜变量，必须满足两个关键且往往相互冲突的约束：**(1)** 它必须封装并保留尽可能多的与智能体j的策略相关的信息；**(2)**它必须采用一种表征格式进行编码，该格式既便于智能体 i 的解码器或行动者网络处理，又能被其解析。因此，选择合适的信号空间

is a pivotal design consideration. In canonical MARL frameworks, this signal is often implicitly defined as the shared interaction history among agents.

In this work, we consider an extreme case in which j's policy encoder ϕ is a source code extractor and i's policy decoder m is a identity function (i.e., $\phi(\pi^j) = \ulcorner \pi^j \urcorner$ and $m = \mathcal{I}$), yielding a perfect observation $\hat{\pi}^j = \ulcorner \pi^j \urcorner$. We denote this setting as the ***unprocessed policy-communication setting***. This setting is possible and can be justified in fully cooperative environments where there is no incentive for deception, or in zero-sum games under self-play (Silver et al., 2017) where the multi-agent system serves as an equilibrium solver.

A key advantage of this setting is that it inherently avoids any information loss that might arise from the communication process and obviates the need for learning a dedicated encoding mechanism. This setting also ***avoids the policy drift and recursive reasoning challenges in opponent modeling***, since agents can accurately obtain each other's policies through communication before acting, rather than having to guess.

The critical drawback, however, is that this approach places a substantial, perhaps prohibitive, parsing burden on the ego agent, which must now interpret the source code of policy directly:

$$a_t^i \sim \pi^i \left(\cdot \mid h_t, \ulcorner \boldsymbol{\pi}^{-i} \urcorner \right).$$

This means that π^i will take other $\ulcorner \boldsymbol{\pi}^{-i} \urcorner$ as input. If all agents' policies are implemented by neural networks as in MARL, this means that π^i's neural network must parse the entire neural network of π^j (i.e., its complete parameter vector θ^j) as input. This leads to two insurmountable obstacles: **(1)** ***Curse of Dimensionality.*** A modern policy network can have millions or billions of parameters. The requirement for π^i's input layer to accommodate and process this magnitude of parameters is computationally prohibitive. It also creates a recursive loop: π^i must grow to parse π^j, and π^j must in turn grow to parse the now-larger π^i. **(2)** ***Curse of Representation.*** This is not just a problem of dimensionality, but of representation. The parameter vector θ^j is a brittle and non-unique representation of the policy function. A neural network's function is invariant to permutations of its hidden nodes; different random initializations lead to functionally similar policies with drastically different parameter vectors. π^i would need to learn a complex, permutation-invariant representation over this unstructured, high-dimensional vector, which is a task for which there is currently no viable solution.

3.2 A Paradigm Shift: Programmatic Policies by LLMs

We posit that, while DRL faces challenges in realizing the unprocessed policy-communication Setting, the use of recently flourishing LLMs to generate code as policies does possess the capacity to handle such policy-input. Its objective is to supplant the opaque, "black box" nature of conventional DRL with policy representations that are structured, interpretable, and more amenable to formal verification.

This paradigm shift is facilitated by the advent of LLMs. LLMs are inherently designed to process structured text and source code. Modern architectures are not only proficient code generators but also feature extensive context windows, capable of managing inputs equivalent to hundreds of thousands of tokens.

This framework directly addresses the aforementioned intractability challenges: **(1)** ***Solving Dimensionality***: While a neural network policy may require millions or billions of parameters to capture complex behaviors, its programmatic equivalent offers a highly compressed, semantically dense representation. Modern LLMs, with their extensive context windows, are capable of processing and reasoning over such complex programmatic structures, thus effectively managing the high intrinsic dimensionality of the policy space. **(2)** ***Solving Representation***: Unlike the unstructured parameter vectors of a neural network, source code is a semantically rich and structured artifact. This representation aligns precisely with the data modalities that LLMs are trained to comprehend, analyze, and generate, enabling high-level reasoning about, and manipulation of, the policy itself.

3.3 Refined Formalization: Meta-Programming and Mathematical Operators

We leverage this new paradigm to reformalize the best-response computation. Mathematically, this process is most precisely described using the terminology of functional analysis. The space of all

是设计中的一项关键考量。在典型的多智能体强化学习框架中，该信号通常被隐式定义为智能体之间的共享交互历史。

在本研究中，我们考虑一种极端情况，其中 j的策略编码器 ϕ 是源代码提取器，而 i的策略解码器 m 是恒等函数（即，$\phi(\pi^j) = \ulcorner \pi^j \urcorner$ 且 $m = \mathcal{I}$），从而得到完美观测$\hat{}\pi^j = \ulcorner \pi^j \urcorner$。我们将这一设定称为未处理策略通信设定。在不存在欺骗动机的完全合作环境中，或在多智能体系统充当均衡求解器的自我博弈零和博弈中（Silver 等人，2017年），这一设定是可行且合理的。

这一设定的一个关键优势在于，它从根本上避免了通信过程可能造成的任何信息损失，也无需学习专门的编码机制。这一设定还避免了对手建模中的策略漂移和递归推理难题，因为智能体能够在行动前通过通信准确获取彼此的策略，而不必进行猜测。

然而，这种方法的关键缺陷在于，它给自我智能体带来了沉重、甚至可能难以承受的解析负担，因为自我智能体现在必须直接解读策略的源代码：

$$a_t^i \sim \pi^i \left(\cdot \mid h_t, \ulcorner \boldsymbol{\pi}^{-i} \urcorner \right).$$

这意味着 π^i 将以其他 $\ulcorner \boldsymbol{\pi}^{-i} \urcorner$ 作为输入。如果所有智能体的策略都像多智能体强化学习中那样由神经网络实现，那么这意味着 π^i的神经网络必须解析作为输入的π^j 的整个神经网络（即其完整的参数向量 θ^j）。这带来了两个无法克服的障碍：**（1）维度灾难。**现代策略网络可能拥有数百万乃至数十亿个参数。要求 π^i的输入层容纳并处理如此规模的参数，在计算上是难以承受的。这还会形成一个递归循环： π^i 必须扩大规模才能解析 π^j，而π^j 又必须相应扩大规模，才能解析如今规模更大的 π^i。**（2）表征灾难。**这不仅是维度问题，也是表征问题。参数向量 θ^j 是策略函数的一种脆弱且不唯一的表征。神经网络的函数在其隐藏节点置换后保持不变；不同的随机初始化会产生功能相似、但参数向量截然不同的策略。 π^i 需要在这个非结构化的高维向量上学习复杂的置换不变表征，而这项任务目前尚无可行的解决方案。

3.2 范式转变：由大语言模型生成程序化策略

我们认为，尽管深度强化学习在实现未经处理的策略通信设定方面面临挑战，但利用近来蓬勃发展的大语言模型生成代码即策略，确实具备处理此类策略输入的能力。其目标是以结构化、可解释且更易于进行形式化验证的策略表征，取代传统深度强化学习不透明的“黑箱”特性。

大语言模型的出现推动了这一范式转变。大语言模型在设计上就具备处理结构化文本和源代码的能力。现代架构不仅是出色的代码生成器，还拥有超大的上下文窗口，能够处理相当于数十万词元的输入。

这一框架直接应对了上述难解性挑战：**(1)** 解决维度问题：神经网络策略可能需要数百万甚至数十亿个参数来捕捉复杂行为，而其程序化等价形式则提供了一种高度压缩、语义密集的表示。现代大语言模型凭借其超大的上下文窗口，能够处理此类复杂的程序化结构并对其进行推理，从而有效应对策略空间的高内在维度。**(2)** 解决表示问题：与神经网络的非结构化参数向量不同，源代码是一种语义丰富的结构化产物。这种表示恰好契合大语言模型在训练中学习理解、分析和生成的数据模态，使其能够对策略本身进行高层推理和操作。

3.3 精细化形式化：元编程与数学算子

我们利用这一新范式重新形式化最优响应计算。从数学角度看，使用泛函分析的术语能够最精确地描述这一过程。所有

policies Π constitutes a function space. An *operator* maps one function space to another; a canonical example is the differential operator D, which maps a function f to its derivative f', i.e., $D(f) = f'$.

Following this definition, our proposed process (denoted as φ^i) is formally a **best-response operator**. It constitutes a mapping $\varphi^i : \{0,1\}^* \to \{0,1\}^*$ that takes an opponent policy function as input and yields a new ego-agent policy function as output. This operator, φ^i, thus computes the "best response" to an input policy function.

From an implementation perspective, this abstract operator φ^i is realized through a **meta-programming** interpreter. Meta-programming is formally defined as the act of a program treating other programs as its data; in this context, it manifests as a code-input, code-output process. This operation is implemented by querying a LLM. Agent i provides its opponent's policy source code $\boldsymbol{\pi}^{-i}$ as input (i.e., as part of the prompt), and the LLM, embodying the operator φ, generates the best-response policy source code π^i as output. Policy adaptation is thus rendered as an interpretable meta-programming task.

Formally, given opponents' policies π^{-i}, the ego agent will take an action as follows:

$$
\begin{aligned}
&\ulcorner \boldsymbol{\pi}^{-i} \urcorner = m\left(\phi\left(\boldsymbol{\pi}^{-i}\right)\right), \\
&\ulcorner \pi^{i} \urcorner = \varphi^i(\ulcorner \boldsymbol{\pi}^{-i} \urcorner \mid \text{prompt}), \\
&\pi^i = \text{CodeInterpreter}(\ulcorner \pi^{i} \urcorner), \\
&a^i \sim \pi^i,
\end{aligned}
$$

where φ^i is implemented by an LLM in our method. Note that the policy of ego agent π^i does not take opponents' source code $\ulcorner \boldsymbol{\pi}^{-i} \urcorner$ as inputs, but it still has the dependency since the generation of π^i is dependent on $\ulcorner \boldsymbol{\pi}^{-i} \urcorner$.

While any generated π^i constitutes a valid response, it is not inherently a best response; we therefore seek the optimal π^i by optimizing the operator φ^i through prompt refinement. Crucially, since we optimize the operator specifically for a fixed input $\boldsymbol{\pi}^{-i}$, this constitutes a **point-wise** optimization of the best-response mapping.

3.4 Learning

We optimize the LLM best-response operators φ in prompt space. This optimization is driven by structured feedback losses and can be automated using gradient-based methods adapted for text. A notable framework in this domain is `TextGrad` (Yuksekgonul et al., 2024),which enables automatic differentiation through text-based models by conceptualizing the text generation process as a computational graph. This allows for the definition of textual feedback and the backpropagation of gradients to update the LLM's guiding prompts or parameters. Once we rigorously define the loss functions, the optimization of the operator φ^i can proceed automatically.

Textual Loss Functions. The feedback provided to the operator φ^i is structured around two primary loss signals: **(1)** *Unit Test Feedback.* A significant challenge in this paradigm is ensuring the syntactic and runtime correctness of the generated policy code. To mitigate this, we employ an automated unit testing phase. The ego policy code $\pi^i = \text{CodeInterpreter}(\ulcorner \pi^{i} \urcorner)$ is instantiated and executed within a sandboxed environment. If the code produces any runtime exceptions (e.g., syntax errors, out-of-bounds access) or fails predefined assertions, the resulting error trace and stack information are captured. This error data serves as a direct, high-priority feedback signal, prompting the LLM operator φ^i to debug and revise the code, akin to automated debugging workflows. **(2)** *Utility Feedback.* The central objective is to ensure the generated policy π^i constitutes a best response to the opponents' policies $\boldsymbol{\pi}^{-i}$. This is evaluated via utility feedback. The policy π^i is executed against $\boldsymbol{\pi}^{-i}$ in the game environment for several episodes. The complete episode trajectory, including all states, joint actions, and rewards for both agents $(s_t, a_t^i, \boldsymbol{a}_t^{-i}, r_t^i, \boldsymbol{r}_t^{-i})_{t=0}^{T}$, is collected. This detailed trajectory information is provided as feedback to the operator φ^i. This allows the operator to assess the performance of its generated policy and refine its meta-programming logic to maximize the expected utility $J^i(\pi^i, \boldsymbol{\pi}^{-i})$.

The overall algorithm follows an iterative best-response dynamic (Robinson, 1951), but lifted to the operator space. The computation of the best response for each agent is not an atomic operation but

策略 Π 构成一个函数空间。算子 将一个函数空间映射到另一个函数空间；一个典型例子是微分算子 D，它将函数 f映射到其导数 f'，即 $D(f) = f'$。

根据这一定义，我们提出的过程（记为 φ^i）在形式上是一个**最优响应算子**。它构成一个映射 φ^i： $\{0,1\}^* \to \{0,1\}^*$，以对手策略函数为输入，并输出一个新的自我智能体策略函数。因此，该算子 φ^i 计算的是输入策略函数的“最优响应”。

从实现角度来看，这一抽象算子 φ^i 通过一个**元编程**解释器来实现。元编程在形式上被定义为程序将其他程序视为自身数据的行为；在此情境中，它表现为一个以代码为输入、以代码为输出的过程。该操作通过查询大语言模型来实现。智能体 i 将其对手的策略源代码 $\boldsymbol{\pi}^{-i}$ 作为输入（即提示词的一部分），而充当算子 φ 的大语言模型则生成最优响应策略源代码 π^i 作为输出。由此，策略适应被转化为一项可解释的元编程任务。

形式上，给定对手策略 π^{-i}，自我智能体将按如下方式采取动作：

$$
\begin{aligned}
&\ulcorner \boldsymbol{\pi}^{-i} \urcorner = m\left(\phi\left(\boldsymbol{\pi}^{-i}\right)\right), \\
&\ulcorner \pi^{i} \urcorner = \varphi^i(\ulcorner \boldsymbol{\pi}^{-i} \urcorner \mid \text{prompt}), \\
&\pi^i = \text{CodeInterpreter}(\ulcorner \pi^{i} \urcorner), \\
&a^i \sim \pi^i,
\end{aligned}
$$

其中， φ^i 在我们的方法中由大语言模型实现。注意，自我智能体的策略 π^i 并不以对手的源代码 $\ulcorner \boldsymbol{\pi}^{-i} \urcorner$ 作为输入，但仍存在依赖关系，因为π^i 的生成依赖于 $\ulcorner \boldsymbol{\pi}^{-i} \urcorner$。

虽然生成的任何 π^i 都构成有效响应，但它本身并不一定是最优响应；因此，我们通过提示词优化来优化算子 φ^i ，以寻求最优的 π^i 。关键在于，由于我们专门针对固定输入 $\boldsymbol{\pi}^{-i}$优化该算子，因此这构成了对最优响应映射的**逐点**优化。

3.4 学习

我们在提示词空间中优化大语言模型最优响应算子 φ 。这一优化由结构化反馈损失驱动，并可使用针对文本调整的基于梯度的方法实现自动化。该领域的一个代表性框架是 `TextGrad` （Yuksekgonul 等人，2024年），它通过将文本生成过程概念化为计算图，实现贯穿文本模型的自动微分。这使得我们能够定义文本反馈，并通过梯度的反向传播来更新大语言模型的引导提示词或参数。一旦我们严格定义了损失函数，算子 φ^i 的优化便可自动进行。

文本损失函数。提供给算子 φ^i 的反馈围绕两种主要损失信号组织：**(1)**单元测试反馈。该范式的一项重大挑战是确保生成的策略代码的语法与运行时正确性。为缓解这一问题，我们采用自动化单元测试阶段。自身策略代码 $\pi^i = \text{CodeInterpreter}(\ulcorner \pi^{i} \urcorner)$ 在沙箱环境中被实例化并执行。如果代码产生任何运行时异常（例如语法错误、越界访问）或未通过预定义断言，则会捕获由此产生的错误追踪和堆栈信息。这些错误数据作为直接且高优先级的反馈信号，提示大语言模型算子 φ^i 调试并修改代码，类似于自动化调试工作流。**(2)** 效用反馈。核心目标是确保生成的策略 π^i 构成对手策略 $\boldsymbol{\pi}^{-i}$的最优响应。这通过效用反馈来评估。策略 π^i 在博弈环境中与$\boldsymbol{\pi}^{-i}$ 进行多个回合的对抗。系统会收集完整的回合轨迹，包括所有状态、联合动作以及两个智能体的奖励 $(s_t, a_t^i, \boldsymbol{a}_t^{-i}, r_t^i, \boldsymbol{r}_t^{-i})_{t=0}^{T}$。这些详细的轨迹信息作为反馈提供给算子 φ^i。这使算子能够评估其生成策略的表现，并改进其元编程逻辑，以最大化期望效用 $J^i(\pi^i, \boldsymbol{\pi}^{-i})$。

整体算法遵循迭代最优响应动态（Robinson，1951年），但将其提升至算子空间。对每个智能体的最优响应的计算并非原子操作，而是

Algorithm 1: Programmatic Iterated Best Response (PIBR)

Input: Game G, Agents $A = \{A^0, A^1\}$, Outer loops K, Inner loops T
Output: Final policy profile $\boldsymbol{\pi}^* = (\pi^{0,*}, \pi^{1,*})$

// Initialize policies and history
$\pi^0 \leftarrow$ InitialPolicy(); $\pi^1 \leftarrow$ InitialPolicy()
$\mathcal{H} \leftarrow \emptyset$ // History of (profile, social welfare)
$i \leftarrow 0$ // Agent index
// Main training phase (Outer Loop)
for $k = 1$ **to** K **do**
  $\pi^{-i} \leftarrow \pi^{1-i}$
  // Compute BR via operator optimization (Inner Loop)
  $\pi^i \leftarrow \text{ComputeBestResponse}(A^i, G, \pi^{-i}, T)$
  $\boldsymbol{\pi}_k \leftarrow (\pi^0, \pi^1)$
  $J_{sw}(\boldsymbol{\pi}_k) \leftarrow \text{EvaluateSocialWelfare}(G, \boldsymbol{\pi}_k)$
  $\mathcal{H} \leftarrow \mathcal{H} \cup \{(\boldsymbol{\pi}_k, J_{sw}(\boldsymbol{\pi}_k))\}$
  $i \leftarrow 1 - i$ // Alternate agent
// Select best profile from history
$(\boldsymbol{\pi}^*, J^*) \leftarrow \arg\max_{(\boldsymbol{\pi}, J) \in \mathcal{H}} J$
return $\boldsymbol{\pi}^*$

Procedure *ComputeBestResponse*(A^i, G, π^{-i}, T)
  $\varphi^i \leftarrow A^i$.operator // Get optimizable operator
  Opt $\leftarrow \text{TextGradOptimizer}(\varphi^i)$
  for $t = 1$ **to** T **do**
    Opt.zero_grad()
    $\ulcorner \pi_t^i \urcorner = \varphi^i\left(\ulcorner \pi^{-i} \urcorner \mid \text{prompt}(A^i, G, t, T)\right)$ // Forward pass
    // Compute composite loss
    $(\pi_t^i, \mathcal{L}_{\text{test}}) \leftarrow \text{ComputeUnitTestLoss}(\ulcorner \pi_t^i \urcorner)$
    $\mathcal{L}_{\text{utility}} \leftarrow -J(G, \pi_t^i, \pi^{-i})$ $\mathcal{L} \leftarrow \mathcal{L}_{\text{test}} + \mathcal{L}_{\text{utility}}$
    $\mathcal{L}$.backward()
    Opt.step()
  return π_t^i

an inner optimization loop guided by feedback. This two-agent version of this process is formally described in Algorithm 1.

4 Experiments

All experiments were powered by OpenAI's `o4-mini` model (OpenAI, 2025), as it provides a particularly effective balance between cost efficiency and code generation performance. The complete codebase, experimental results, and execution logs will be released as open source after the paper is accepted.

4.1 Tasks

Matrix Games. We consider three representative coordination matrix games that vary in their payoff structures and coordination difficulty. In each game, two agents simultaneously choose an action (row and column respectively), and both receive the same payoff corresponding to the selected matrix entry. **(a)** The `Vanilla Coordination Game` is a simple setting with three pure coordination equilibria of different rewards, testing whether agents can align on the optimal coordination point. **(b)** The `Climbing Game` (Claus & Boutilier, 1998) introduces strong miscoordination penalties and uneven reward gradients, making it a challenging variant where naive exploration can easily lead to suboptimal equilibria. Finally, **(c)** the `Penalty Game` (Claus & Boutilier, 1998) includes negative

算法 1： 程序化迭代最优响应（PIBR）

输入： 博弈 G，智能体 $A = \{A^0, A^1\}$，外层循环 K，内层循环 T
输出： 最终策略组合 $\boldsymbol{\pi}^* = (\pi^{0,*}, \pi^{1,*})$

// 初始化策略和历史
$\pi^0 \leftarrow$ InitialPolicy(); $\pi^1 \leftarrow$ InitialPolicy()
$\mathcal{H} \leftarrow \emptyset$// （策略组合，社会福利）的历史
$i \leftarrow 0$// 智能体索引
// 主训练阶段（外层循环）
对于 $k = 1$ **到** K **执行**
  $\pi^{-i} \leftarrow \pi^{1-i}$
  // 通过算子优化计算最优响应（内层循环）
  $\pi^i \leftarrow$ 计算最优响应(A^i, G, π^{-i}, T)
  $\boldsymbol{\pi}_k \leftarrow (\pi^0, \pi^1)$
  $J_{sw}(\boldsymbol{\pi}_k) \leftarrow$ 评估社会福利$(G, \boldsymbol{\pi}_k)$
  $\mathcal{H} \leftarrow \mathcal{H} \cup \{(\boldsymbol{\pi}_k, J_{sw}(\boldsymbol{\pi}_k))\}$
  $i \leftarrow 1 - i$ // 交替切换智能体
// 从历史中选择最佳策略组合
$(\boldsymbol{\pi}^*, J^*) \leftarrow \arg\max_{(\boldsymbol{\pi}, J) \in \mathcal{H}} J$
返回 $\boldsymbol{\pi}^*$

过程 计算最优响应(A^i, G, π^{-i}, T)
  $\varphi^i \leftarrow A^i$.算子 // 获取可优化算子
  优化器 $\leftarrow \text{TextGradOptimizer}(\varphi^i)$
  对于 $t = 1$**到** T**执行**
    Opt.zero_grad()
    $\ulcorner \pi_t^i \urcorner = \varphi^i\left(\ulcorner \pi^{-i} \urcorner \mid \text{提示词}(A^i, G, t, T)\right)$ // 前向传播
    // 计算复合损失
    $(\pi_t^i, \mathcal{L}_{\text{test}}) \leftarrow$ 计算单元测试损失$(\ulcorner \pi_t^i \urcorner)$
    $\mathcal{L}_{\text{utility}} \leftarrow -J(G, \pi_t^i, \pi^{-i})$ $\mathcal{L} \leftarrow \mathcal{L}_{\text{test}} + \mathcal{L}_{\text{utility}}$
    $\mathcal{L}$.backward()
    Opt.step()
  return π_t^i

由反馈引导的内层优化循环。算法 1 对该过程的双智能体版本给出了形式化描述。

4 实验

所有实验均采用 OpenAI 的 `o4-mini` 模型（OpenAI，2025年），因为该模型在成本效益与代码生成性能之间实现了尤为出色的平衡。完整的代码库、实验结果和执行日志将在论文被接收后开源发布。

4.1 任务

矩阵博弈。我们考虑三种具有代表性的协调矩阵博弈，它们的收益结构和协调难度各不相同。在每种博弈中，两个智能体同时选择一个动作（分别对应行和列），并获得所选矩阵元素对应的相同收益。**(a)** 其中，基础协调博弈是一种简单的设定，包含三个奖励不同的纯协调均衡，用于测试智能体能否在最优协调点上达成一致。**(b)**而攀登博弈（Claus & Boutilier，1998年）引入了严厉的失协调惩罚和不均匀的奖励梯度，使其成为一种具有挑战性的变体，朴素探索很容易导致次优均衡。最后，**(c)** 在惩罚博弈（Claus & Boutilier，1998年）中，存在负的

diagonal payoffs ($p < 0$) that discourage coordination on certain actions, requiring agents to learn to avoid detrimental equilibria while still maximizing joint payoff. Together, these games provide a controlled yet diverse testbed for studying coordination behaviors in multi-agent learning.

	a^j		
a^i	2	0	0
	0	1	0
	0	0	3

(a) Vanilla Coordination Game

	a^j		
a^i	11	−30	0
	−30	7	0
	0	6	5

(b) Climbing Game

	a^j		
a^i	p	0	10
	0	2	0
	10	0	p

(c) Penalty Game ($p < 0$)

Figure 1: Three 3×3 coordination matrix games used in our experiments. Each game defines payoffs for two agents (row agent's action $a^i \in \{0, 1, 2\}$, column agent's action $a^j \in \{0, 1, 2\}$): after they simultaneously choose a row and a column action, both receive the utility indicated by the selected matrix entry.

Level-Based Foraging. Level-Based Foraging (LBF) (Christianos et al., 2020) is a grid-world environment for studying multi-agent coordination. Agents are to collect food items to gain rewards. Each agent can take one of six actions: `[stay_idle, move_up, move_down, move_left, move_right, load_food]`. In the original environment, food items can only be collected when the sum of the levels of agents performing the `load` action adjacent to the food meets or exceeds the food's level, making it a mixed-motive task. [1] We consider a fully cooperative variant here: each food is assigned a level such that it always requires both agents to perform the load action simultaneously to be collected. Rewards are shared between agents proportionally to their skill levels and normalized. An episode ends when all foods are collected or the maximum length is reached. We further modify the environment by introducing a terminal penalty that is inversely proportional to the episode length, making the improvement of food collection efficiency the optimization objective for the agents.

4.2 Results

The experimental results, summarized in Figure 2, demonstrate the effectiveness of PIBR across distinct game types. In these plots, the horizontal axis represents the sequential optimization steps, where each step corresponds to a policy update step driven by textual gradients. The vertical axis denotes the Social Welfare, calculated as the sum of cumulative rewards obtained by the agents. For every optimization step, we sample multiple episodes to evaluate the current policy profile; these individual episodes are visualized as scattered points, while the dashed line traces the mean reward trajectory across optimization steps. The absence of data points at certain optimization steps indicates that the generated policy code contained syntax or runtime errors, failing to execute in the environment. **(1) Results of Coordination Games:** across all three coordination tasks, the agents exhibit rapid and robust convergence to the global optima. In `Vanilla Coordination` and `Climbing Game`, the social welfare instantaneously stabilizes at the optimal values of 6.0 and 22.0, respectively. In the `Penalty Game`, despite the risk of substantial penalties, the agents successfully coordinate on the optimal equilibrium within a single update step. **(2) Results of Cooperative Foraging:** In this complex grid-world task, while individual samples occasionally approach the empirical optimum (≈ 0.554). The mean trajectory exhibits significant variance and instability. The performance drop in later stages indicates the challenge of maintaining consistent coordination in complicated tasks. Following Line 11 of Algorithm 1, the procedure returns the agents' strategies from the 16-th optimization step, since it yields the highest social welfare. We recognize that the current results are not yet fully stabilized, and further improvements to the method's robustness are under active development.

[1] E.g., collecting Level 1 foods in the grid-world is competitive among Level 1 agents, but they must cooperate to collect Level 2 foods.

对角线收益（$p < 0$），这会抑制智能体在某些动作上进行协调，要求智能体学会在最大化联合收益的同时避免不利的均衡。总体而言，这些博弈为研究多智能体学习中的协调行为提供了一个可控且多样化的测试平台。

	a^j		
a^i	2	0	0
	0	1	0
	0	0	3

(a) 基础协调博弈

	a^j		
a^i	11	−30	0
	−30	7	0
	0	6	5

(b) 攀登博弈

	a^j		
a^i	p	0	10
	0	2	0
	10	0	p

(c) 惩罚博弈（$p < 0$）

图1：实验中使用的三种 3×3 协调矩阵博弈。每种博弈都为两个智能体定义了收益（行智能体的动作 $a^i \in \{0, 1, 2\}$，列智能体的动作 $a^j \in \{0, 1, 2\}$）：两个智能体同时分别选择行和列动作后，均获得所选矩阵元素所指示的效用。

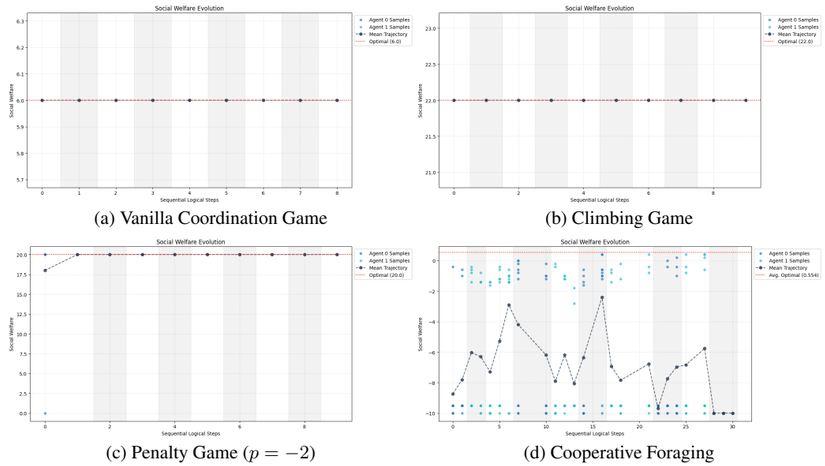
基于等级的觅食。基于等级的觅食（基于等级的觅食）（Christianos 等人，2020年）是一个用于研究多智能体协调的网格世界环境。智能体通过收集食物项来获得奖励。每个智能体可以采取以下六种动作之一：`[保持_空闲、移动_向上、移动_向下、移动_向左、移动_向右、装载_食物]`。在原始环境中，只有当食物相邻位置上执行装载动作的智能体的等级之和达到或超过该食物的等级时，才能收集该食物项，因此这是一项混合动机任务。 [1] 这里我们考虑一种完全合作变体：为每个食物设定等级，使其始终需要两个智能体同时执行装载动作才能被收集。奖励按智能体的技能等级比例分配，并进行归一化。当所有食物被收集完毕或达到最大回合长度时，回合结束。我们进一步修改了环境，引入与回合长度成反比的终止惩罚，从而将提高食物收集效率作为智能体的优化目标。

4.2 结果

图2汇总了实验结果，展示了 PIBR 在不同博弈类型中的有效性。在这些图中，横轴表示顺序优化步骤，每一步对应一个由文本梯度驱动的策略更新步骤。纵轴表示社会福利，其计算方式为各智能体获得的累积奖励之和。在每个优化步骤中，我们采样多个回合来评估当前策略组合；这些单独的回合以散点形式呈现，而虚线则描绘了各优化步骤上的平均奖励轨迹。某些优化步骤缺少数据点，表明生成的策略代码包含语法或运行时错误，未能在环境中执行。**（1）协调博弈的结果：**在全部三个协调任务中，智能体均表现出向全局最优解的快速且稳健的收敛。在基础协调博弈和攀登博弈中，社会福利立即稳定在最优值，分别为 6.0 和 22.0。在惩罚博弈中，尽管存在遭受重大惩罚的风险，智能体仍在单次更新步骤内成功协调至最优均衡。**（2）合作型博弈的结果**

觅食：在这一复杂的网格世界任务中，尽管个别样本偶尔接近经验最优值（≈ 0.554），但平均轨迹仍表现出显著的方差和不稳定性。后期的性能下降表明，在复杂任务中维持一致协调具有挑战性。根据算法 1 的第 11 行，该过程返回第 16 次优化步骤得到的智能体策略，因为这些策略带来了最高的社会福利。我们认识到，目前的结果尚未完全稳定，正在积极改进该方法的鲁棒性。

[1] 例如，在网格世界中，一级智能体在收集一级食物时存在竞争，但它们必须合作才能收集二级食物。



(a) Vanilla Coordination Game (b) Climbing Game

(c) Penalty Game ($p = -2$) (d) Cooperative Foraging

Figure 2: Experimental results.

5 Conclusion

In this work, we address the challenge of non-stationarity in solving Markov games by proposing a paradigm shift from opaque neural policies to interpretable programmatic policies, thereby overcoming the "representational bottleneck" that hinders direct conditioning on opponent strategies. We reformulate the learning problem by lifting optimization from the policy space to the operator space, employing LLMs as point-wise best-response operators that dynamically synthesize policy code based on the opponent's source code. We operationalize this via the Programmatic Iterated Best Response algorithm, which optimizes the meta-programming logic through textual gradients derived from both game utility and runtime unit test feedback. The proposed method seeks program equilibria rather than black-box Nash equilibria, facilitating a broader scope of cooperation. Our experiments in coordination matrix games and Level-Based Foraging demonstrate that PIBR effectively bridges the gap between theoretical Program Equilibrium and modern learning, enabling the discovery of robust, coordinated strategies through the exchange and analysis of executable code.

References

Elif Akata, Lion Schulz, Julian Coda-Forno, Seong Joon Oh, Matthias Bethge, and Eric Schulz. Playing repeated games with large language models. *Nature Human Behaviour*, pp. 1–11, 2025.

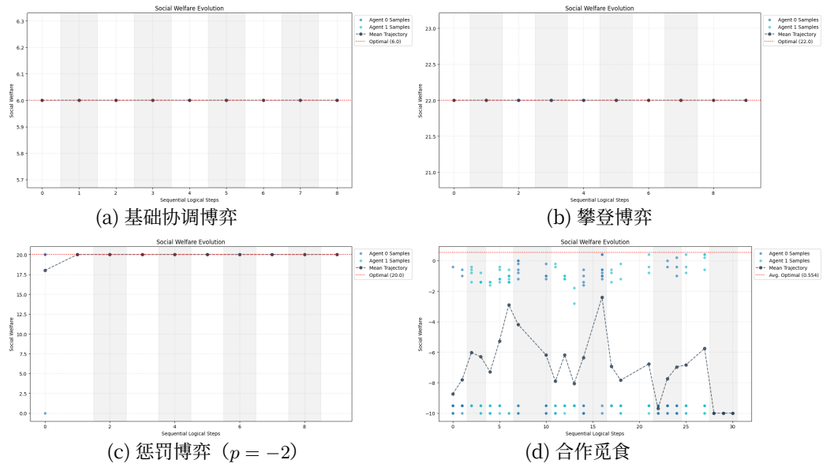
Ashwath Vaithinathan Aravindan, Zhisheng Tang, and Mayank Kejriwal. Code-driven planning in grid worlds with large language models. *arXiv preprint arXiv:2505.10749*, 2025.

Yoram Bachrach, Edan Toledo, Karen Hambardzumyan, Despoina Magka, Martin Josifoski, Minqi Jiang, Jakob Foerster, Roberta Raileanu, Tatiana Shavrina, Nicola Cancedda, et al. Combining code generating large language models and self-play to iteratively refine strategies in games. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI-25*, pp. 10999–11003, 2025.

Mihaly Barasz, Paul Christiano, Benja Fallenstein, Marcello Herreshoff, Patrick LaVictoire, and Eliezer Yudkowsky. Robust cooperation in the prisoner's dilemma: Program equilibrium via provability logic. *arXiv preprint arXiv:1401.5577*, 2014.

Ulrich Berger. Brown's original fictitious play. *Journal of Economic Theory*, 135(1):572–578, 2007.

R Selten Bielefeld. Reexamination of the perfectness concept for equilibrium points in extensive games. In *Models of strategic rationality*, pp. 1–31. Springer, 1988.



(a) 基础协调博弈 (b) 攀登博弈

(c) 惩罚博弈（$p = -2$） (d) 合作觅食

图 2：实验结果。

5 结论

在本研究中，我们针对求解马尔可夫博弈时的非平稳性挑战，提出从不透明的神经网络策略转向可解释程序化策略的范式转变，从而克服阻碍直接以对手策略为条件的“表征瓶颈”。我们将优化从策略空间提升至算子空间，以此重新表述学习问题，并将大语言模型用作逐点最佳响应算子，根据对手的源代码动态合成策略代码。我们通过程序化迭代最佳响应算法实现这一方法，利用从博弈效用和运行时单元测试反馈中提取的文本梯度来优化元编程逻辑。所提出的方法寻求程序均衡，而非黑箱纳什均衡，从而拓展了合作的范围。我们在协调矩阵博弈和基于等级的觅食中的实验表明，PIBR 有效弥合了理论上的程序均衡与现代学习之间的鸿沟，使智能体能够通过交换和分析可执行代码，发现鲁棒且协调的策略。

参考文献

Elif Akata、Lion Schulz、Julian Coda-Forno、Seong Joon Oh、Matthias Bethge 和 Eric Schulz。使用大语言模型进行重复博弈。*自然·人类行为*，第 1–11 页，2025年。

Ashwath Vaithinathan Aravindan、Zhisheng Tang 和 Mayank Kejriwal。利用大语言模型在网格世界中进行代码驱动规划。*arXiv* 预印本 *arXiv:2505.10749*，2025年。

Yoram Bachrach、Edan Toledo、Karen Hambardzumyan、Despoina Magka、Martin Josifoski、Minqi Jiang、Jakob Foerster、Roberta Raileanu、Tatiana Shavrina、Nicola Cancedda 等。结合代码生成大语言模型与自我博弈，迭代优化博弈策略。载于*第三十四届国际人工智能联合会议论文集，国际人工智能联合会议-25*，第 10999–11003 页，2025年。

Mihaly Barasz、Paul Christiano、Benja Fallenstein、Marcello Herreshoff、Patrick LaVictoire 和埃利泽·尤德考斯基。囚徒困境中的鲁棒合作：通过可证性逻辑实现程序均衡。*arXiv* 预印本 *arXiv:1401.5577*，2014年。

Ulrich Berger。布朗原始虚拟博弈。*经济理论杂志*，135(1):572–578，2007年。

R Selten Bielefeld。对扩展式博弈中均衡点完美性概念的重新审视。载于*战略理性模型*，第1–31页。施普林格，1988年。

Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, Ryan Julian, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on robot learning*, pp. 287–318. PMLR, 2023.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023.

Filippos Christianos, Lukas Schäfer, and Stefano V Albrecht. Shared experience actor-critic for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, 2020.

Caroline Claus and Craig Boutilier. The dynamics of reinforcement learning in cooperative multiagent systems. *AAAI/IAAI*, 1998(746-752):2, 1998.

Anthony Costarelli, Mat Allen, Roman Hauksson, Grace Sodunke, Suhas Hariharan, Carlson Cheng, Wenjie Li, Joshua Clymer, and Arjun Yadav. Gamebench: Evaluating strategic reasoning abilities of llm agents. *arXiv preprint arXiv:2406.06613*, 2024.

Andrew Critch. Parametric bounded l\" ob's theorem and robust cooperation of bounded agents. *arXiv preprint arXiv:1602.04184*, 2016.

Andrew Critch, Michael Dennis, and Stuart Russell. Cooperative and uncooperative institution designs: Surprises and problems in open-source game theory. *arXiv preprint arXiv:2208.07006*, 2022.

Anthony DiGiovanni and Jesse Clifton. Commitment games with conditional information disclosure. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pp. 5616–5623, 2023.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Forty-first International Conference on Machine Learning*, 2023.

Gabriele Farina. Lecture 16: Markov (aka stochastic) games. Lecture notes for 6.S890 Topics in Multiagent Learning, November 2024. URL https://www.mit.edu/~gfarina/2024/6S890f24_L16_sg/L16.pdf. Accessed: 2025-11-16.

Arlington M Fink. Equilibrium in a stochastic n-person game. *Journal of science of the hiroshima university, series ai (mathematics)*, 28(1):89–93, 1964.

Jakob N Foerster, Richard Y Chen, Maruan Al-Shedivat, Shimon Whiteson, Pieter Abbeel, and Igor Mordatch. Learning with opponent-learning awareness. *arXiv preprint arXiv:1709.04326*, 2017.

Lance Fortnow. Program equilibria and discounted computation time. In *Proceedings of the 12th Conference on Theoretical Aspects of Rationality and Knowledge*, pp. 128–133, 2009.

Drew Fudenberg and Jean Tirole. *Game theory*. MIT press, 1991.

Piotr J Gmytrasiewicz and Prashant Doshi. A framework for sequential planning in multi-agent settings. *Journal of Artificial Intelligence Research*, 24:49–79, 2005.

Ian Goodfellow, Yoshua Bengio, Aaron Courville, and Yoshua Bengio. *Deep learning*. MIT press Cambridge, 2016.

Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. In *IJCAI*, 2024.

Shanshan Han, Qifan Zhang, Yuhang Yao, Weizhao Jin, and Zhaozhuo Xu. Llm multi-agent systems: Challenges and open problems. *arXiv preprint arXiv:2402.03578*, 2024.

He He, Jordan Boyd-Graber, Kevin Kwok, and Hal Daumé III. Opponent modeling in deep reinforcement learning. In *International conference on machine learning*, pp. 1804–1813. PMLR, 2016.

Anthony Brohan、Yevgen Chebotar、Chelsea Finn、Karol Hausman、Alexander Herzog、Daniel Ho、Julian Ibarz、Alex Irpan、Eric Jang、Ryan Julian等。按我能做的做，而非按我说的做：将语言落地于机器人可供性。载于机器人学习会议，第287–318页。PMLR，2023年。Wenhu Chen、Xueguang Ma、Xinyi Wang和William W Cohen。思维程序提示：在数值推理任务中将计算与推理解耦。机器学习研究汇刊，2023年。Filippos Christianos、Lukas Schäfer和Stefano V Albrecht。用于多智能体强化学习的共享经验演员-评论家算法。载于神经信息处理系统进展，2020年。Caroline Claus和Craig Boutilier。合作型多智能体系统中的强化学习动态。AAAI/IAAI，1998年(746-752):2，1998年。Anthony Costarelli、Mat Allen、Roman Hauksson、Grace Sodunke、Suhas Hariharan、Carlson Cheng、Wenjie Li、Joshua Clymer和Arjun Yadav。Gamebench：评估大语言模型智能体的策略推理能力。arXiv预印本arXiv:2406.06613，2024年。Andrew Critch。参数化有界L\" öb定理与有界智能体的鲁棒合作。arXiv预印本arXiv:1602.04184，2016年。Andrew Critch、Michael Dennis和斯图尔特·罗素。合作与非合作制度设计：开源博弈论中的意外发现与问题。arXiv预印本arXiv:2208.07006，2022年。Anthony DiGiovanni和Jesse Clifton。具有条件信息披露的承诺博弈。载于AAAI人工智能会议论文集，第37卷，第5616–5623页，2023年。Yilun Du、Shuang Li、Antonio Torralba、Joshua B Tenenbaum和Igor Mordatch。通过多智能体辩论提升语言模型的事实准确性与推理能力。载于第四十一届国际机器学习大会，2023年。Gabriele Farina。第16讲：马尔可夫博弈（又称随机博弈）。6.S890多智能体学习专题课程讲义，2024年11月。网址：https://www.mit.edu/~gfarina/2024/ 6S890f24_L16_sg/L16.pdf。访问日期：2025年11月16日。Arlington M Fink。随机 n人博弈中的均衡。广岛大学科学学报，AI系列（数学），28(1):89–93，1964年。Jakob N Foerster、Richard Y Chen、Maruan Al-Shedivat、Shimon Whiteson、Pieter Abbeel和Igor Mordatch。对手学习感知学习。arXiv预印本arXiv:1709.04326，2017年。Lance Fortnow。程序均衡与折扣计算时间。载于第十二届理性与知识理论方面会议论文集，第128–133页，2009年。德鲁·弗登伯格和让·梯若尔。博弈论。麻省理工学院出版社，1991年。Piotr J Gmytrasiewicz和Prashant Doshi。多智能体环境下的序贯规划框架。人工智能研究期刊，24:49–79，2005年。伊恩·古德费洛、约书亚·本吉奥、Aaron Courville和约书亚·本吉奥。深度学习。麻省理工学院出版社，剑桥，2016年。Taicheng Guo、Xiuying Chen、Yaqi Wang、Ruidi Chang、Shichao Pei、Nitesh V Chawla、Olaf Wiest和Xiangliang Zhang。基于大语言模型的多智能体：进展与挑战综述。载于国际人工智能联合会议，2024年。Shanshan Han、Qifan Zhang、Yuhang Yao、Weizhao Jin和Zhaozhuo Xu。大语言模型多智能体系统：挑战与开放问题。arXiv预印本arXiv:2402.03578，2024年。He He、Jordan Boyd-Graber、Kevin Kwok和Hal Daumé III。深度强化学习中的对手建模。载于国际机器学习会议，第1804–1813页。PMLR，2016年。

Pablo Hernandez-Leal, Michael Kaisers, Tim Baarslag, and Enrique Munoz De Cote. A survey of learning in multiagent environments: Dealing with non-stationarity. *arXiv preprint arXiv:1707.09183*, 2017.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2023.

Vijay Konda and John Tsitsiklis. Actor-critic algorithms. *Advances in neural information processing systems*, 12, 1999.

Vojtech Kovarik, Caspar Oesterheld, and Vincent Conitzer. Game theory with simulation of other players. *arXiv preprint arXiv:2305.11261*, 2023.

Patrick LaVictoire, Benja Fallenstein, Eliezer Yudkowsky, Mihaly Barasz, Paul Christiano, and Marcello Herreshoff. Program equilibrium in the prisoner's dilemma via löb's theorem. In *AAAI workshop on multiagent interaction without prior coordination*, 2014.

Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for" mind" exploration of large language model society. *Advances in Neural Information Processing Systems*, 36:51991–52008, 2023.

Wenhao Li, Yue Lin, Xiangfeng Wang, Bo Jin, Hongyuan Zha, and Baoxiang Wang. Verbalized bayesian persuasion. *arXiv preprint arXiv:2502.01587*, 2025.

Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9493–9500. IEEE, 2023.

Yaobo Liang, Chenfei Wu, Ting Song, Wenshan Wu, Yan Xia, Yu Liu, Yang Ou, Shuai Lu, Lei Ji, Shaoguang Mao, Yun Wang, Linjun Shou, Ming Gong, and Nan Duan. Taskmatrix.ai: Completing tasks by connecting foundation models with millions of apis. *Intelligent Computing*, 3:0063, 2024.

Yue Lin, Wenhao Li, Hongyuan Zha, and Baoxiang Wang. Information design in multi-agent reinforcement learning. *Advances in Neural Information Processing Systems*, 36:25584–25597, 2023.

Yue Lin, Shuhui Zhu, William A Cunningham, Wenhao Li, Pascal Poupart, Hongyuan Zha, and Baoxiang Wang. Bayesian persuasion as a bargaining game. *arXiv preprint arXiv:2506.05876*, 2025.

Michael L Littman. Markov games as a framework for multi-agent reinforcement learning. In *Machine learning proceedings 1994*, pp. 157–163. Elsevier, 1994.

Max Liu, Chan-Hung Yu, Wei-Hsu Lee, Cheng-Wei Hung, Yen-Chun Chen, and Shao-Hua Sun. Synthesizing programmatic reinforcement learning policies with large language model guided search. In *The Thirteenth International Conference on Learning Representations*, 2025.

Shaoteng Liu, Haoqi Yuan, Minda Hu, Yanwei Li, Yukang Chen, Shu Liu, Zongqing Lu, and Jiaya Jia. Rl-gpt: Integrating reinforcement learning and code-as-policy. *Advances in Neural Information Processing Systems*, 37:28430–28459, 2024.

Ryan Lowe, Yi I Wu, Aviv Tamar, Jean Harb, OpenAI Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in neural information processing systems*, 30, 2017.

Hanbin Luo, Jianxin Wu, Jiajing Liu, and Maxwell Fordjour Antwi-Afari. Large language model-based code generation for the control of construction assembly robots: A hierarchical generation approach. *Developments in the Built Environment*, 19:100488, 2024.

Pablo Hernandez-Leal、Michael Kaisers、Tim Baarslag和Enrique Munoz De Cote。多智能体环境中的学习综述：应对非平稳性。*arXiv*预印本 *arXiv:1707.09183*，2017年。Sirui Hong、Mingchen Zhuge、Jonathan Chen、Xiawu Zheng、Yuheng Cheng、Jinlin Wang、Ceyao Zhang、Zili Wang、Steven Ka Shing Yau、Zijuan Lin等。MetaGPT：多智能体协作框架的元编程。载于第十二届国际学习表征会议，2023年。Vijay Konda和John Tsitsiklis。演员-评论家算法。神经信息处理系统进展，12，1999年。Vojtech Kovarik、Caspar Oesterheld和Vincent Conitzer。具有其他参与者模拟的博弈论。*arXiv*预印本 *arXiv:2305.11261*，2023年。Patrick LaVictoire、Benja Fallenstein、埃利泽·尤德考斯基、Mihaly Barasz、Paul Christiano和Marcello Herreshoff。通过勒布定理实现囚徒困境中的程序均衡。载于*AAAI*无事先协调的多智能体交互研讨会，2014年。Guohao Li、Hasan Hammoud、Hani Itani、Dmitrii Khizbullin和Bernard Ghanem。CAMEL：用于探索大语言模型社会“心智”的通信智能体。神经信息处理系统进展，36:51991–52008，2023年。Wenhao Li、Yue Lin、Xiangfeng Wang、Bo Jin、Hongyuan Zha和Baoxiang Wang。言语化贝叶斯说服。*arXiv*预印本 *arXiv:2502.01587*，2025年。Jacky Liang、Wenlong Huang、Fei Xia、Peng Xu、Karol Hausman、Brian Ichter、Pete Florence和Andy Zeng。代码即策略：用于具身控制的语言模型程序。载于*2023*年*IEEE*国际机器人与自动化会议（*ICRA*），第9493–9500页。电气与电子工程师协会，2023年。Yaobo Liang、Chenfei Wu、Ting Song、Wenshan Wu、Yan Xia、Yu Liu、Yang Ou、Shuai Lu、Lei Ji、Shaoguang Mao、Yun Wang、Linjun Shou、Ming Gong和Nan Duan。TaskMatrix.AI：通过将基础模型与数百万个应用程序编程接口连接来完成任务。智能计算，3:0063，2024年。Yue Lin、Wenhao Li、Hongyuan Zha和Baoxiang Wang。多智能体强化学习中的信息设计。神经信息处理系统进展，36:25584–25597，2023年。Yue Lin、Shuhui Zhu、William A Cunningham、Wenhao Li、Pascal Poupart、Hongyuan Zha和Baoxiang Wang。作为议价博弈的贝叶斯劝说。*arXiv*预印本 *arXiv:2506.05876*，2025年。Michael L Littman。作为多智能体强化学习框架的马尔可夫博弈。载于*1994*年机器学习会议论文集，第157–163页。爱思唯尔，1994年。Max Liu、Chan-Hung Yu、Wei-Hsu Lee、Cheng-Wei Hung、Yen-Chun Chen和Shao-Hua Sun。通过大语言模型引导搜索合成程序化强化学习策略。载于第十三届国际学习表征会议，2025年。Shaoteng Liu、Haoqi Yuan、Minda Hu、Yanwei Li、Yukang Chen、Shu Liu、Zongqing Lu和Jiaya Jia。RL-GPT：融合强化学习与代码即策略。神经信息处理系统进展，37:28430–28459，2024年。Ryan Lowe、Yi I Wu、Aviv Tamar、Jean Harb、OpenAI Pieter Abbeel和Igor Mordatch。面向合作竞争混合环境的多智能体演员-评论家方法。神经信息处理系统进展，30，2017年。Hanbin Luo、Jianxin Wu、Jiajing Liu和Maxwell Fordjour Antwi-Afari。基于大语言模型的建筑装配机器人控制代码生成：一种分层生成方法。建成环境发展，19:100488，2024年。

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36:46534–46594, 2023.

John F Nash Jr. Equilibrium points in n-person games. *Proceedings of the national academy of sciences*, 36(1):48–49, 1950.

Caspar Oesterheld and Vincent Conitzer. Safe pareto improvements for delegated game playing. *Autonomous Agents and Multi-Agent Systems*, 36(2):46, 2022.

OpenAI. Introducing openai o3 and o4-mini, 2025. URL https://openai.com/index/introducing-o3-and-o4-mini/.

Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pp. 1–22, 2023.

Michael Peters and Balázs Szentes. Definable and contractible contracts. *Econometrica*, 80(1): 363–411, 2012.

Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15174–15186, 2024.

Neil Rabinowitz, Frank Perbet, Francis Song, Chiyuan Zhang, SM Ali Eslami, and Matthew Botvinick. Machine theory of mind. In *International conference on machine learning*, pp. 4218–4227. PMLR, 2018.

Tabish Rashid, Mikayel Samvelyan, Christian Schroeder De Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. Monotonic value function factorisation for deep multi-agent reinforcement learning. *Journal of Machine Learning Research*, 21(178):1–51, 2020.

Julia Robinson. An iterative method of solving a game. *Annals of mathematics*, 54(2):296–301, 1951.

Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using llm agents as research assistants. *arXiv preprint arXiv:2501.04227*, 2025.

Lloyd S Shapley. Stochastic games. *Proceedings of the national academy of sciences*, 39(10): 1095–1100, 1953.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.

David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *nature*, 550(7676):354–359, 2017.

Swadesh Sistla and Max Kleiman-Weiner. Evaluating llms in open-source games. *arXiv preprint arXiv:2512.00371*, 2025a.

Swadesh Sistla and Max Kleiman-Weiner. Evaluating llms in open-source games. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025b.

Chan Hee Song, Jiaman Wu, Clayton Washington, Brian M Sadler, Wei-Lun Chao, and Yu Su. Llm-planner: Few-shot grounded planning for embodied agents with large language models. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 2998–3009, 2023.

Xinyuan Sun, Davide Crapis, Matt Stephenson, Barnabé Monnot, Thomas Thiery, and Jonathan Passerat-Palmbach. Cooperative ai via decentralized commitment devices. *arXiv preprint arXiv:2311.07815*, 2023.

Aman Madaan、Niket Tandon、Prakhar Gupta、Skyler Hallinan、Luyu Gao、Sarah Wiegreffe、Uri Alon、Nouha Dziri、Shrimai Prabhumoye、Yiming Yang等。Self-refine：通过自我反馈进行迭代改进。神经信息处理系统进展，36:46534–46594，2023年。小约翰·F·纳什。多人博弈中的均衡点。美国国家科学院院刊，36(1):48–49，1950年。Caspar Oesterheld和Vincent Conitzer。委托博弈中的安全帕累托改进。自主智能体与多智能体系统，36(2):46，2022年。OpenAI。介绍OpenAI o3和o4-mini，2025年。网址：https://openai.com/index/introducing-o3-and-o4-mini/。Joon Sung Park、Joseph O'Brien、Carrie Jun Cai、Meredith Ringel Morris、Percy Liang和Michael S Bernstein。生成式智能体：人类行为的交互式拟像。载于第*36*届*ACM*用户界面软件与技术年度研讨会论文集，第1–22页，2023年。Michael Peters和Balázs Szentes。可定义且可契约化的合约。计量经济学，80(1):363–411，2012年。Chen Qian、Wei Liu、Hongzhang Liu、Nuo Chen、Yufan Dang、Jiahao Li、Cheng Yang、Weize Chen、Yusheng Su、Xin Cong等。ChatDev：用于软件开发的通信智能体。载于第*62*届计算语言学协会年会论文集（第*1*卷：长篇论文），第15174–15186页，2024年。Neil Rabinowitz、Frank Perbet、Francis Song、Chiyuan Zhang、SM Ali Eslami和Matthew Botvinick。机器心智理论。载于国际机器学习会议，第4218–4227页。PMLR，2018年。Tabish Rashid、Mikayel Samvelyan、Christian Schroeder De Witt、Gregory Farquhar、Jakob Foerster和Shimon Whiteson。用于深度多智能体强化学习的单调值函数分解。机器学习研究期刊，21(178):1–51，2020年。朱莉娅·罗宾逊。一种求解博弈的迭代方法。数学年刊，54(2):296–301，1951年。Samuel Schmidgall、Yusheng Su、Ze Wang、Ximeng Sun、Jialian Wu、Xiaodong Yu、Jiang Liu、Zicheng Liu和Emad Barsoum。Agent Laboratory：将大语言模型智能体用作研究助理。*arXiv*预印本 *arXiv:2501.04227*，2025年。劳埃德·S·沙普利。随机博弈。美国国家科学院院刊，39(10):1095–1100，1953年。Noah Shinn、Federico Cassano、Ashwin Gopinath、Karthik Narasimhan和Shunyu Yao。Reflexion：具有言语强化学习能力的语言智能体。神经信息处理系统进展，36:8634–8652，2023年。戴维·西尔弗、Julian Schrittwieser、Karen Simonyan、Ioannis Antonoglou、黄士杰、Arthur Guez、Thomas Hubert、Lucas Baker、Matthew Lai、Adrian Bolton等。不依赖人类知识掌握围棋。自然，550(7676):354–359，2017年。Swadesh Sistla和Max Kleiman-Weiner。在开源博弈中评估大语言模型。*arXiv*预印本 *arXiv:2512.00371*，2025a。Swadesh Sistla和Max Kleiman-Weiner。在开源博弈中评估大语言模型。载于第三十九届神经信息处理系统年度会议，2025b。Chan Hee Song、Jiaman Wu、Clayton Washington、Brian M Sadler、Wei-Lun Chao和Yu Su。LLM-Planner：利用大语言模型为具身智能体进行少样本接地规划。载于*IEEE/CVF*国际计算机视觉大会论文集，第2998–3009页，2023年。Xinyuan Sun、Davide Crapis、Matt Stephenson、Barnabé Monnot、Thomas Thiery和Jonathan Passerat-Palmbach。通过去中心化承诺机制实现合作式人工智能。*arXiv*预印本 *arXiv:2311.07815*，2023年。

Peter Sunehag, Guy Lever, Audrunas Gruslys, Wojciech Marian Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z Leibo, Karl Tuyls, et al. Value-decomposition networks for cooperative multi-agent learning. *arXiv preprint arXiv:1706.05296*, 2017.

Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems*, 12, 1999.

Masayuki Takahashi. Equilibrium points of stochastic non-cooperative n-person games. *Journal of Science of the Hiroshima University, Series AI (Mathematics)*, 28(1):95–99, 1964.

Moshe Tennenholtz. Program equilibrium. *Games and Economic Behavior*, 49(2):363–373, 2004.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. In *Intrinsically-Motivated and Open-Ended Learning Workshop@ NeurIPS2023*, 2023a.

Zihao Wang, Shaofei Cai, Guanzhou Chen, Anji Liu, Xiaojian Shawn Ma, and Yitao Liang. Describe, explain, plan and select: interactive planning with llms enables open-world multi-task agents. *Advances in Neural Information Processing Systems*, 36:34153–34189, 2023b.

Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.

Richard Willis, Yali Du, Joel Z Leibo, and Michael Luck. Will systems of llm agents cooperate: An investigation into a social dilemma. *arXiv preprint arXiv:2501.16173*, 2025.

Yuzhuang Xu, Shuo Wang, Peng Li, Fuwen Luo, Xiaolong Wang, Weidong Liu, and Yang Liu. Exploring large language models for communication games: An empirical study on werewolf. *arXiv preprint arXiv:2309.04658*, 2023.

Jiachen Yang, Ang Li, Mehrdad Farajtabar, Peter Sunehag, Edward Hughes, and Hongyuan Zha. Learning to incentivize other learning agents. *Advances in Neural Information Processing Systems*, 33:15208–15219, 2020.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic "differentiation" via text. *arXiv preprint arXiv:2406.07496*, 2024.

Ceyao Zhang, Kaijie Yang, Siyi Hu, Zihao Wang, Guanghe Li, Yihang Sun, Cheng Zhang, Zhaowei Zhang, Anji Liu, Song-Chun Zhu, et al. Proagent: building proactive cooperative agents with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 17591–17599, 2024.

Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. Gptswarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024.

Peter Sunehag、Guy Lever、Audrunas Gruslys、Wojciech Marian Czarnecki、Vinicius Zambaldi、Max Jaderberg、Marc Lanctot、Nicolas Sonnerat、Joel Z Leibo、Karl Tuyls 等。用于合作式多智能体学习的价值分解网络。*arXiv* 预印本 *arXiv:1706.05296*，2017年。Richard S Sutton、David McAllester、Satinder Singh 和 Yishay Mansour。采用函数逼近的强化学习策略梯度方法。神经信息处理系统进展，12，1999年。Masayuki Takahashi。随机非合作 n人博弈的均衡点。广岛大学科学学报，*AI*系列（数学），28(1):95–99，1964年。Moshe Tennenholtz。程序均衡。博弈与经济行为，49(2):363–373，2004年。Guanzhi Wang、Yuqi Xie、Yunfan Jiang、Ajay Mandlekar、Chaowei Xiao、Yuke Zhu、Linxi Fan 和 Anima Anandkumar。Voyager：基于大语言模型的开放式具身智能体。载于*NeurIPS2023*内在动机与开放式学习研讨会，2023年a。Zihao Wang、Shaofei Cai、Guanzhou Chen、Anji Liu、Xiaojian Shawn Ma 和 Yitao Liang。描述、解释、计划与选择：利用大语言模型的交互式规划实现开放世界多任务智能体。神经信息处理系统进展，36:34153–34189，2023年b。Ronald J Williams。用于联结主义强化学习的简单统计梯度跟随算法。机器学习，8(3):229–256，1992年。Richard Willis、Yali Du、Joel Z Leibo 和 Michael Luck。大语言模型智能体系统会合作吗：一项关于社会困境的研究。*arXiv* 预印本 *arXiv:2501.16173*，2025年。Yuzhuang Xu、Shuo Wang、Peng Li、Fuwen Luo、Xiaolong Wang、Weidong Liu 和 Yang Liu。探索大语言模型在交流博弈中的应用：一项关于狼人杀的实证研究。*arXiv* 预印本 *arXiv:2309.04658*，2023年。Jiachen Yang、Ang Li、Mehrdad Farajtabar、Peter Sunehag、Edward Hughes 和 Hongyuan Zha。学习激励其他学习智能体。神经信息处理系统进展，33:15208–15219，2020年。Mert Yuksekgonul、Federico Bianchi、Joseph Boen、Sheng Liu、Zhi Huang、Carlos Guestrin 和 James Zou。TextGrad：通过文本实现自动“微分”。*arXiv* 预印本 *arXiv:2406.07496*，2024年。Ceyao Zhang、Kaijie Yang、Siyi Hu、Zihao Wang、Guanghe Li、Yihang Sun、Cheng Zhang、Zhaowei Zhang、Anji Liu、朱松纯 等。Proagent：利用大语言模型构建主动合作智能体。载于*AAAI*人工智能会议论文集，第38卷，第17591–17599页，2024年。Mingchen Zhuge、Wenyi Wang、Louis Kirsch、Francesco Faccio、Dmitrii Khizbullin 和 于尔根·施密德胡贝尔。Gptswarm：将语言智能体表示为可优化图。载于第四十一届国际机器学习大会，2024年。

Appendix

Table of Contents

A Related Works **16**
A.1 Opponent Modeling 16
A.2 Commitment 16
A.3 Fictitious Play 17
A.4 LLMs for Multi-Agent Tasks 17
A.5 Code Policies by LLMs 18

B Program Equilibrium in the Prisoner's Dilemma **18**
B.1 Feasibility: Mutual Cooperation via Löb's Theorem 19
B.2 Stability: Robustness against Deviations 19

C Generated Code-Policies **19**
C.1 Generated Code-Policies for the Climbing Game 19
C.2 Generated Code-Policies for the Cooperative Foraging Task 20

D Glossary **25**

A Related Works

A.1 Opponent Modeling

A large body of MARL research can be categorized by how it approaches this modeling function m. **(1)** *Implicit Modeling.* A dominant branch of research, particularly within the CTDE framework, models opponents implicitly. In methods like Value-Decomposition Networks (VDN) (Sunehag et al., 2017), QMIX (Rashid et al., 2020), and MADDPG (Lowe et al., 2017), a centralized critic is provided with global state and action information from all agents during training. This critic implicitly learns the interaction dynamics, conveys information by *parameter sharing*, and uses this knowledge to guide the decentralized actors. The resulting actors themselves do not have an explicit mechanism at execution time to adapt to novel opponent policies not seen during training. **(2)** *Explicit Modeling.* An alternative approach attempts to learn $\hat{\pi}^{-i}$ explicitly. Works like He et al. (2016) design neural architectures aimed explicitly at predicting the opponent's behavior or encoding it into a latent space. While theoretically more adaptive, these methods face the challenge of learning a high-dimensional mapping from history to the policy, which is also a high-dimensional object.

There is also some work that goes a step further by being aware of the other agent's learning dynamics to better predict the ego agent's expected value (Foerster et al., 2017), or even leverages mechanisms to alter the other agent's behavior (Yang et al., 2020; Lin et al., 2023).

A.2 Commitment

A central challenge in committing on other agents' source code is ensuring credible, robust, and non-exploitative cooperation when agents can read, simulate, or reason about each other's programs. Classical program equilibrium (Tennenholtz, 2004) shows that full cooperation in one-shot games is achievable when programs condition on opponents' code, but these early constructions are brittle and depend on exact code matching. Logical approaches using Löb's theorem demonstrate that agents can instead cooperate when they can prove that the opponent will reciprocate, yielding robust program equilibria without requiring identical code (LaVictoire et al., 2014; Barasz et al., 2014), though still vulnerable to issues of exploitation and representational fragility. Follow-up work extends these

附录

目录

A 相关工作 **16**
A.1 对手建模 .16A.2 承诺.16A.3 虚拟博弈 .17A.4 用于多智能体任务的大语言模型 .
17A.5 大语言模型生成的代码策略 .18

B 囚徒困境中的程序均衡 18
B.1 可行性：通过勒布定理实现相互合作 .
19B.2 稳定性：对偏离的鲁棒性 .19

C 生成的代码策略 19C.1 为攀登博弈生成的代码策略 .19C.2 为合作觅食任务生成的代
码策略 .20

D 术语表 **25**

A 相关工作

A.1 对手建模

大量多智能体强化学习研究可根据其实现这一建模功能的方式进行分类m。**(1)** 隐式建模。一个主流研究分支，尤其是在集中式训练、分散式执行框架内，采用隐式方式对对手建模。在价值分解网络（价值分解网络）（Sunehag 等，2017年）、QMIX（Rashid 等，2020年）和多智能体深度确定性策略梯度（Lowe 等，2017年）等方法中，训练期间会向集中式评价器提供全局状态以及所有智能体的动作信息。该评价器隐式地学习交互动态，通过参数共享传递信息，并利用这些知识指导分散式行动器。由此得到的行动器本身在执行时并不具备显式机制，无法适应训练期间未见过的新对手策略。**(2)** 显式建模。另一种方法尝试显式学习 $\hat{}\pi^{-i}$ 。例如，He 等（2016年）的研究设计了专门用于预测对手行为或将其编码到潜在空间中的神经网络架构。尽管这些方法在理论上具有更强的适应性，但它们面临的挑战是学习从历史到策略的高维映射，而策略本身也是一个高维对象。

还有一些研究更进一步，通过感知其他智能体的学习动态来更好地预测自我智能体的期望值（Foerster 等，2017年），甚至利用某些机制来改变其他智能体的行为（Yang 等，2020年；Lin 等，2023年）。

A.2 承诺

基于其他智能体的源代码作出承诺时，一个核心挑战是：当智能体能够读取、模拟或推理彼此的程序时，如何确保可信、鲁棒且非剥削性的合作。经典程序均衡（Tennenholtz，2004年）表明，当程序以对手的代码为条件时，一次性博弈中的完全合作是可以实现的，但这些早期构造十分脆弱，依赖于精确的代码匹配。使用勒布定理的逻辑方法表明，智能体也可以在能够证明对手会予以回报时开展合作，从而无需代码完全相同便可实现鲁棒的程序均衡（LaVictoire 等，2014年；Barasz 等人，2014年），但仍易受到剥削和表示脆弱性问题的影响。后续工作将这些

results to resource-bounded agents via parametric bounded Löb reasoning (Critch, 2016). When agents can simulate each other at a cost, cooperation can emerge when distrust is the only barrier, but simulation can also reduce welfare by enabling new threats (Kovarik et al., 2023). Under incomplete information, conditional information disclosure provides commitment devices that sustain efficient outcomes (DiGiovanni & Clifton, 2023). Similarly, Lin et al. (2025) frame the selection of these information structures as a bargaining game, enabling both sender and receiver agents to commit to signaling schemes that enforce cooperative equilibria. This parallels formal contracting frameworks in which agents write enforceable program-like contracts (Peters & Szentes, 2012). In practice, decentralized cryptographic mechanisms provide commitment infrastructure but introduce new security and integrity risks (Sun et al., 2023). Open-source game theory reveals counterintuitive or unstable outcomes even under full transparency (Critch et al., 2022), highlighting unresolved robustness challenges. Recent work on safe Pareto improvements addresses some of these risks by identifying commitment restrictions that are guaranteed not to harm any party (Oesterheld & Conitzer, 2022), while empirical studies of LLM agents in open-source games show both cooperative and deceptive behaviors, underscoring the need for practical safeguards when commitments rely on program inspection (Sistla & Kleiman-Weiner, 2025b).

A.3 Fictitious Play

Fictitious play (FP) (Robinson, 1951; Berger, 2007) is a classical game-theoretic learning dynamic in which each agent iteratively best-responds to the empirical frequency of others' past actions, often converging to a Nash equilibrium in certain classes of games (Bachrach et al., 2025). FP has been foundational in multi-agent learning and has been extended to richer settings where agents explicitly model or inspect their opponents' decision-making procedures. In the extreme case of open-source or program games (Tennenholtz, 2004), each player submits an algorithm that can read the other's source code, enabling novel equilibria based on mutual transparency (e.g. two programs mutually cooperating in a one-shot dilemma) (Fortnow, 2009). Tennenholtz (2004)'s program equilibrium framework formalized how agents conditioning on opponents' code can achieve such outcomes beyond standard game-theoretic limits. These ideas are relevant to modern AI: LLM-based agents can simulate or infer an opponent's policy or reasoning process, effectively building a theory-of-mind. For instance, prompting an LLM to reason about an opponent's likely action via a social chain-of-thought markedly improves coordination in repeated games (Akata et al., 2025). The intersection of fictitious-play-style adaptation with LLMs' capacity for modeling other agents thus points to a promising avenue for multi-agent systems, where agents dynamically learn and adapt by reasoning about each other's strategies.

A.4 LLMs for Multi-Agent Tasks

Recently, due to the powerful reasoning capabilities and extensive intrinsic knowledge of LLMs, research on LLM-based multi-agent systems has progressed rapidly (Guo et al., 2024; Han et al., 2024). This line of work can be broadly divided into two categories: social simulation and task collaboration. Social simulation research utilizes the natural language interaction capabilities of LLMs to analyze and calibrate emergent behaviors in different social interactions, benchmarking them against human society. Park et al. (2023) develops the first sandbox environment driven by generative LLM agents to simulate a small-scale community of 25 agents and provides a user-interactive interface for engagement. CAMEL (Li et al., 2023) provides an scalable framework to support dialogue-intensive scenarios and keep consistency of agents during role playing, which is suitable for studying conversational behaviors. Xu et al. (2023) investigates emergent strategic behaviors—such as trust, confrontation, camouflage, and leadership—of LLMs in Werewolf, revealing the strong potential of LLMs in communication games. Beyond simple interaction, Li et al. (2025) explore strategic verbal communication, utilizing LLMs to operationalize Bayesian persuasion and influence recipients' beliefs through natural language.

In task collaboration, the objective is to develop algorithmic frameworks that coordinate multiple LLMs to complete complex tasks, including natural-language tasks and text-based games (Costarelli et al., 2024; Schmidgall et al., 2025; Qian et al., 2024). MetaGPT (Hong et al., 2023) incorporates standardized operating procedures (SOPs) into multi-LLM workflows to improve coordination and the accuracy of intermediate processes, which decomposes software development task dinto multiple subtasks and roles. ProAgent (Zhang et al., 2024) proposes a modular planning framework

结果通过参数化有界洛布推理推广至资源受限智能体（Critch，2016年）。当智能体能够付出一定成本来模拟彼此时，如果不信任是唯一障碍，合作便可能涌现，但模拟也可能因促成新的威胁而降低福利（Kovarik 等人，2023年）。在不完全信息条件下，条件信息披露提供了能够维持高效结果的承诺装置（DiGiovanni & Clifton，2023年）。类似地，Lin 等（2025年）将这些信息结构的选择建模为议价博弈，使发送方和接收方智能体都能承诺采用促成合作均衡的信号传递方案。这与形式化契约框架相呼应，在此类框架中，智能体编写可强制执行的类程序契约（Peters & Szentes，2012年）。在实践中，去中心化密码学机制提供了承诺基础设施，但也引入了新的安全性与完整性风险（Sun 等人，2023年）。开源博弈论揭示，即使在完全透明的条件下，也会出现反直觉或不稳定的结果（Critch 等人，2022年），凸显了尚未解决的鲁棒性挑战。近期关于安全帕累托改进的研究通过识别保证不会损害任何一方的承诺限制，应对了其中一些风险（Oesterheld& Conitzer，2022年）；与此同时，针对开源博弈中大语言模型智能体的实证研究表明，它们既会表现出合作行为，也会表现出欺骗行为，这凸显了在承诺依赖程序检查时采取实际保障措施的必要性（Sistla & Kleiman-Weiner，2025b）。

A.3 虚拟博弈

虚拟博弈（虚拟博弈）（Robinson，1951年；Berger，2007年）是一种经典的博弈论学习动态，其中每个智能体迭代地对其他智能体过去动作的经验频率作出最优响应，在某些类别的博弈中通常会收敛到纳什均衡（Bachrach 等人，2025年）。虚拟博弈一直是多智能体学习的基础，并已扩展到更丰富的情境，其中智能体会显式建模或检查对手的决策程序。在开源博弈或程序博弈这一极端情形中（Tennenholtz，2004年），每位参与者提交一个能够读取对方源代码的算法，从而实现基于相互透明的新型均衡（例如，两个程序在单次困境中相互合作）（Fortnow，2009年）。Tennenholtz（2004年）的程序均衡框架形式化地说明了：以对手代码为条件进行决策的智能体，如何实现超越标准博弈论局限的这类结果。这些思想与现代人工智能密切相关：基于大语言模型的智能体能够模拟或推断对手的策略或推理过程，从而有效构建心智理论。例如，通过提示让大语言模型借助社会思维链推理对手可能采取的动作，可以显著改善重复博弈中的协调（Akata 等人，2025年）。因此，将虚拟博弈式适应与大语言模型对其他智能体的建模能力相结合，为多智能体系统指明了一条前景广阔的路径：智能体通过推理彼此的策略，动态地学习和适应。

A.4 用于多智能体任务的大语言模型

近年来，凭借大语言模型强大的推理能力和丰富的内在知识，基于大语言模型的多智能体系统研究取得了快速进展（Guo 等人，2024年；Han 等人，2024年）。这一研究方向大致可分为两类：社会模拟与任务协作。社会模拟研究利用大语言模型的自然语言交互能力，分析和校准不同社会互动中的涌现行为，并以人类社会为基准进行比较。Park 等人（2023年）开发了首个由生成式大语言模型智能体驱动的沙盒环境，用于模拟一个由25个智能体组成的小型社区，并提供了可供用户参与的交互界面。CAMEL（Li 等人，2023年）提供了一个可扩展框架，以支持对话密集型场景，并在角色扮演过程中保持智能体的一致性，适用于研究对话行为。Xu 等人（2023年）研究了大语言模型在狼人杀中涌现出的策略行为，如信任、对抗、伪装和领导力，揭示了大语言模型在交流博弈中的巨大潜力。除简单交互之外，Li 等人（2025年）探索了策略性言语交流，利用大语言模型将贝叶斯劝说付诸实践，并通过自然语言影响接收者信念。

在任务协作中，目标是开发算法框架，协调多个大语言模型完成复杂任务，包括自然语言任务和文本游戏（Costarelli 等人，2024年；Schmidgall 等人，2025年；Qian 等人，2024年）。MetaGPT（Hong 等人，2023年）将标准化操作程序（SOPs）融入多大语言模型工作流，以改善协调并提高中间过程的准确性，将软件开发任务分解为多个子任务和角色。ProAgent（Zhang 等人，2024年）提出了一种模块化规划框架

to facilitate coordination with arbitrary opponents, including belief, plan, memory, self-reflection, and action. GPTswarm (Zhuge et al., 2024) represents multi-agent LLM systems as computational graphs, where each LLM agent is represented as a node and the communication among agents is represented as the edges. This flexible structure enables ensembling diverse LLM functions and optimizing workflow topologies. Multi-Agent Debate Du et al. (2023) draws inspiration from debate and voting mechanisms in human society, using multiple rounds of reasoning and opinion exchange to improve accuracy on mathematical and QA problems.

However, these approaches often rely on natural-language dialogue to exchange information and intent, or treat LLM as executable policies with huge parameters. This leads to a lack of game-theoretic guarantees and commitment constraints in communication, which can introduce ambiguity and hallucinations into the messages. Furthermore, the large parameter space poses severe challenges for policy optimization and coordination. Our approach effectively improves policy interpretability and reduces the search space by expressing the policy as executable code and communicating it explicitly.

A.5 Code Policies by LLMs

A growing body of work investigates using large language models to generate executable programs that function directly as control policies, offering interpretability and compositional structure absent in neural actors. Code as Policies (CaP) (Liang et al., 2023) generates Python programs that compose motion primitives and conditional rules, producing explicit and adaptable robot-control policies. Code-driven planning (Aravindan et al., 2025) formulates policy generation as program synthesis, producing loops and state-handling logic that act as executable decision policies in grid-world environments. Hierarchical program generation (Luo et al., 2024) creates both high-level and low-level control code for construction-robot tasks, encoding sequencing and safety constraints as explicit policy logic. RL-GPT (Liu et al., 2024) integrates reinforcement learning with LLM-generated program structures, allowing reward-driven refinement of high-level policy code. Voyager (Wang et al., 2023a) continuously synthesizes and improves executable skill programs, treating each program as a reusable policy component for lifelong embodied learning. Reflexion (Shinn et al., 2023) iteratively updates code-like decision rules after execution failures, forming a self-improving program-structured policy. Self-Refine (Madaan et al., 2023) applies critique-and-correct refinement to improve generated code, yielding progressively stronger programmatic decision rules. Program-of-Thought prompting (Chen et al., 2023) generates executable intermediate programs that act as explicit policies in sequential decision-making tasks. LLM-Planner (Song et al., 2023) synthesizes executable planning scripts whose encoded procedures serve as interpretable multi-step policies. SayCan (Brohan et al., 2023) outputs structured action sequences resembling policy scripts, using affordance constraints to guide task-level decision making. DEPS (Wang et al., 2023b) produces decomposed control programs that function as hierarchical policies for navigation and manipulation tasks. LLM-Guided Search (LLM-GS) (Liu et al., 2025) uses LLM priors to guide the synthesis of discrete programmatic RL policies, though DSL compliance remains a core limitation. TaskMatrix.AI (Liang et al., 2024) generates API-calling scripts that orchestrate multi-tool operations, serving as high-level executable policies. LLM-driven strategy generation (Willis et al., 2025) translates LLM-produced strategies into executable code for iterated social-dilemma settings, enabling systematic evaluation of cooperative and adversarial policies.

B Program Equilibrium in the Prisoner's Dilemma

In this appendix, we explicitly derive the Program Equilibrium for the Prisoner's Dilemma (PD) using the formalism of Provability Logic. Recall that the standard PD payoffs satisfy $r_T > r_R > r_P > r_S$.

We analyze the stability of the `FairBot` program, denoted as π_{FB}. The source code of π_{FB} is defined as follows:

$$\pi_{\text{FB}}(\ulcorner \pi^{-i} \urcorner) = \begin{cases} \text{“Cooperation”} & \text{if } \mathcal{T} \vdash \pi^{-i}(\ulcorner \pi_{\text{FB}} \urcorner) = \text{“Cooperation”} \\ \text{“Defect”} & \text{otherwise} \end{cases} \tag{1}$$

where $\mathcal{T} \vdash \mathcal{P}$ denotes that the statement $\mathcal{P}$ is provable in the base theory $\mathcal{T}$ (e.g., Peano Arithmetic), denoted by $\Box\mathcal{P}$ in modal logic.

以促进与任意对手的协调，其中包括信念、计划、记忆、自我反思和动作。GPTswarm（Zhuge 等人，2024年）将多智能体大语言模型系统表示为计算图，其中每个大语言模型智能体表示为一个节点，智能体之间的通信表示为边。这种灵活的结构支持集成多种大语言模型功能并优化工作流拓扑。多智能体辩论 Du 等人（2023年）从人类社会的辩论和投票机制中汲取灵感，通过多轮推理和意见交流，提高数学问题和问答问题的回答准确性。

然而，这些方法往往依赖自然语言对话来交换信息与意图，或将大语言模型视为具有庞大参数量的可执行策略。这导致通信缺乏博弈论保证和承诺约束，可能使消息产生歧义与幻觉。此外，庞大的参数空间给策略优化和协调带来了严峻挑战。我们的方法将策略表示为可执行代码并显式传达，从而有效提高策略可解释性并缩小搜索空间。

A.5 大语言模型生成的代码策略

越来越多的研究探索利用大语言模型生成可直接充当控制策略的可执行程序，从而提供神经网络行动者所缺乏的可解释性与组合结构。代码即策略（CaP）（Liang 等人，2023年）生成组合运动基元与条件规则的 Python 程序，构建显式且适应性强的机器人控制策略。代码驱动规划（Aravindan 等人，2025年）将策略生成表述为程序合成，生成循环与状态处理逻辑，使其在网格世界环境中充当可执行决策策略。分层程序生成（Luo 等人，2024年）为建筑机器人任务创建高层与底层控制代码，将顺序编排与安全约束编码为显式策略逻辑。RL-GPT（Liu 等人，2024年）将强化学习与大语言模型生成的程序结构相结合，使高层策略代码能够进行奖励驱动优化。Voyager（Wang 等人，2023年a）持续合成并改进可执行技能程序，将每个程序视为终身具身学习中的可复用策略组件。Reflexion（Shinn 等人，2023年）在执行失败后迭代更新类代码的决策规则，形成自我改进的程序结构化策略。Self-refine（Madaan 等人，2023年）采用批评与纠正式优化来改进生成的代码，产生逐步增强的程序化决策规则。思维程序提示（Chen 等人，2023年）生成可执行中间程序，使其在序贯决策任务中充当显式策略。LLM-Planner（Song 等人，2023年）合成可执行规划脚本，其中编码的过程充当可解释的多步策略。SayCan（Brohan 等人，2023年）输出类似策略脚本的结构化动作序列，利用可供性约束指导任务级决策。DEPS（Wang 等人，2023年b）生成分解式控制程序，使其在导航与操作任务中充当分层策略。大语言模型引导搜索（LLM-GS）（Liu 等人，2025年）利用大语言模型先验指导离散程序化强化学习策略的合成，但领域特定语言合规性仍是其核心局限。TaskMatrix.AI（Liang 等人，2024年）生成协调多工具操作的 API调用脚本，充当高层可执行策略。大语言模型驱动的策略生成（Willis 等人，2025年）将大语言模型生成的策略转换为用于迭代社会困境情境的可执行代码，从而支持对合作与对抗策略进行系统评估。

B 囚徒困境中的程序均衡

在本附录中，我们使用可证性逻辑的形式体系，显式推导囚徒困境中的程序均衡。回顾一下，标准囚徒困境的收益满足 $r_T > r_R > r_P > r_S$。

我们分析 FairBot 程序的稳定性，将该程序记为 π_{FB}。π_{FB} 的源代码定义如下：

$$\pi_{\text{FB}}(\ulcorner \pi^{-i} \urcorner) = \begin{cases} \text{“Cooperation”} & \text{if } \mathcal{T} \vdash \pi^{-i}(\ulcorner \pi_{\text{FB}} \urcorner) = \text{“Cooperation”} \\ \text{“Defect”} & \text{otherwise} \end{cases} \tag{1}$$

其中，$\mathcal{T} \vdash \mathcal{P}$ 表示命题 $\mathcal{P}$ 在基础理论 $\mathcal{T}$（例如皮亚诺算术）中可证，在模态逻辑中记为 $\Box\mathcal{P}$。

To demonstrate that the profile $(\pi_{\text{FB}}, \pi_{\text{FB}})$ constitutes a Nash equilibrium, we examine both on-path feasibility and off-path stability.

B.1 Feasibility: Mutual Cooperation via Löb's Theorem

Consider the case where two agents both submit π_{FB}. Let $\mathcal{P}$ be the sentence asserting that `FairBot` cooperates against itself:

$$\mathcal{P} := (\pi_{\text{FB}}(\ulcorner \pi_{\text{FB}} \urcorner) = \text{“Cooperation”})$$

By the definition of the program, π_{FB} cooperates if and only if it finds a proof that its opponent (which is itself) cooperates. Therefore, we have the fixed-point equivalence:

$$\mathcal{T} \vdash (\mathcal{P} \leftrightarrow \Box\mathcal{P}) \tag{2}$$

To prove that cooperation actually occurs ($\mathcal{P}$), we invoke **Löb's Theorem**, which states that for any sentence A, if $\mathcal{T} \vdash (\Box A \rightarrow A)$, then $\mathcal{T} \vdash A$.

We verify the condition for Löb's Theorem as follows:

1. From the program definition (2), we have the implication $\mathcal{T} \vdash (\Box\mathcal{P} \rightarrow \mathcal{P})$. *Intuitive explanation of the above implication:* Inside the formal system, if we hypothetically assume that a proof of cooperation exists ($\Box\mathcal{P}$), the program's condition is satisfied, and it will output “Cooperation” ($\mathcal{P}$).
2. Since $\mathcal{T} \vdash (\Box\mathcal{P} \rightarrow \mathcal{P})$ holds, Löb's Theorem implies $\mathcal{T} \vdash \mathcal{P}$.

Thus, the system proves that `FairBot` cooperates. Assuming the formal system $\mathcal{T}$ is sound (i.e., proven statements are true in reality), both programs output “Cooperation”, yielding the payoff r_R.

B.2 Stability: Robustness against Deviations

We check for Nash equilibrium by considering a unilateral deviation. Suppose player $-i$ adheres to π_{FB}. We analyze whether player i can gain by deviating to an arbitrary program $\pi' \in \Pi_{\text{prog}}$.

The equilibrium payoff is $u^i(\pi_{\text{FB}}, \pi_{\text{FB}}) = r_R$.

Case 1: Cooperative Deviation. Suppose player i submits a program π' such that $\mathcal{T} \vdash \pi'(\ulcorner \pi_{\text{FB}} \urcorner) =$ “Cooperation”. Since π_{FB} searches for precisely this proof, π_{FB} will cooperate. The outcome is (C, C), yielding payoff r_R. The deviation does not strictly increase the payoff.

Case 2: Defecting Deviation. Suppose player i submits a program π' that actually defects against π_{FB}, i.e., $\pi'(\ulcorner \pi_{\text{FB}} \urcorner) =$ “Defect”. `FairBot` will only cooperate if it finds a proof of π' cooperating:

$$\Box(\pi'(\ulcorner \pi_{\text{FB}} \urcorner) = \text{“Cooperation”})$$

Assuming the base theory $\mathcal{T}$ is **consistent**, it cannot prove a false statement. Since π' actually defects, the statement “π' cooperates” is false. Therefore, no proof exists ($\neg\Box$(“π' cooperates”)). Consequently, the condition for π_{FB} fails, and π_{FB} outputs “Defect”.

The resulting outcome is (D, D), yielding payoff r_P. Since $r_R > r_P$ in the Prisoner's Dilemma, this deviation results in a strictly lower payoff.

$$u^i(\pi_{\text{FB}}, \pi_{\text{FB}}) \geq u^i(\pi', \pi_{\text{FB}}) \quad \forall \pi'$$

Thus, $(\pi_{\text{FB}}, \pi_{\text{FB}})$ is a Nash equilibrium.

C Generated Code-Policies

C.1 Generated Code-Policies for the Climbing Game

Here we show the result from a single run of the task `Climbing Game`. The detailed description is in Section 4.1.

Agent 0's final policy is shown as below.

为证明策略组合（π_{FB}， π_{FB}）构成纳什均衡，我们同时考察路径上可行性和路径外稳定性。

B.1 可行性：通过勒布定理实现相互合作

考虑两个智能体均提交 π_{FB} 的情况。令 $\mathcal{P}$ 为断言 FairBot 与自身对弈时会合作的语句：

$$\mathcal{P} := (\pi_{\text{FB}}(\ulcorner \pi_{\text{FB}} \urcorner) = \text{“Cooperation”})$$

根据程序的定义， π_{FB} 当且仅当找到其对手（即其自身）会合作的证明时才会合作。因此，我们得到以下不动点等价关系：

$$\mathcal{T} \vdash (\mathcal{P} \leftrightarrow \Box\mathcal{P}) \tag{2}$$

为了证明合作确实会发生（$\mathcal{P}$），我们援引**勒布定理**，该定理指出，对于任意语句 A，若 $\mathcal{T} \vdash$（$\Box A \rightarrow A$），则 $\mathcal{T} \vdash A$。

我们按如下方式验证勒布定理的条件：

1. 根据程序定义（2），我们得到蕴涵 $\mathcal{T} \vdash$（$\Box\mathcal{P} \rightarrow \mathcal{P}$）。对上述蕴涵的直观解释：在形式系统内，如果我们假设存在合作的证明（$\Box\mathcal{P}$），则程序的条件得到满足，程序将输出“合作”（$\mathcal{P}$）。2. 由于 $\mathcal{T} \vdash$（$\Box\mathcal{P} \rightarrow \mathcal{P}$）成立，勒布定理蕴涵 $\mathcal{T} \vdash \mathcal{P}$。

因此，该系统证明了 FairBot 会合作。假设形式系统 $\mathcal{T}$是可靠的（即已证明的陈述在现实中为真），则两个程序都会输出“合作”，产生收益 r_R。

B.2 稳定性：对偏离的鲁棒性

我们通过考虑单方面偏离来检验纳什均衡。假设参与者 $-i$ 遵循π_{FB}。我们分析参与者 i 能否通过偏离至任意程序 $\pi' \in \Pi_{\text{prog}}$而获益。

均衡收益为 $u^i(\pi_{\text{FB}}, \pi_{\text{FB}}) = r_R$。

情形 1：合作性偏离。假设参与者 i 提交一个程序 π'，使得 $\mathcal{T} \vdash \pi'(\ulcorner \pi_{\text{FB}} \urcorner) =$“合作”。由于 π_{FB} 搜索的正是这一证明， π_{FB} 将会合作。结果为 (C, C)，产生收益 r_R。该偏离并未严格提高收益。

情形 2：背叛性偏离。假设参与者 i 提交一个程序 π'，该程序实际上会背叛π_{FB}，即 $\pi'(\ulcorner \pi_{\text{FB}} \urcorner) =$“背叛”。FairBot 只有在找到 π' 会合作的证明时才会合作：

$$\Box(\pi'(\ulcorner \pi_{\text{FB}} \urcorner) = \text{“Cooperation”})$$

假设基础理论 $\mathcal{T}$是一**致的**，那么它就无法证明一个假命题。由于 π' 实际上选择背叛，命题“π' 合作”为假。因此，不存在证明（$\neg\Box$（“π'合作”））。由此， π_{FB} 的条件不成立， π_{FB} 输出“背叛”。

最终结果为（D, D），收益为 r_P。由于在囚徒困境中 $r_R > r_P$，这种偏离会导致收益严格降低。

$$u^i(\pi_{\text{FB}}, \pi_{\text{FB}}) \geq u^i(\pi', \pi_{\text{FB}}) \quad \forall \pi'$$

因此，（$\pi_{\text{FB}}, \pi_{\text{FB}}$）是一个纳什均衡。

C 生成的代码策略

C.1 攀登博弈中生成的代码策略

这里展示任务的一次运行结果，该任务为攀登博弈。详细说明见第4.1节。

智能体0的最终策略如下所示。

```python
from typing import List, Dict, Any, Callable, TypedDict, Union, Optional

class MatrixGameHistory(TypedDict):
    state: List[int]
    action: List[List[int]]

def history_dependent_policy_0(game_history: MatrixGameHistory) -> List[float]:
    # The opponent always plays action 0 with probability 1.0 (committed cooperation).
    # Best response is to also play action 0 to secure the 11-point payoff.
    # I am matching your cooperation. Please keep playing 0.
    return [1.0, 0.0, 0.0]
```

Agent 1's final policy is shown as below.

```python
from typing import List, TypedDict

class MatrixGameHistory(TypedDict):
    state: List[int]
    action: List[List[int]]

def history_dependent_policy_1(game_history: MatrixGameHistory) -> List[float]:
    # Opponent commits to playing action 0 with probability 1.0.
    # Best response to secure the 11-point payoff is to also play action 0.
    # I am committing to cooperation. Please match me.
    return [1.0, 0.0, 0.0]
```

C.2 Generated Code-Policies for the Cooperative Foraging Task

Here we show the result from a single run of the task `Foraging-5x5-2p-2f-coop-v3`. This corresponds to a Level-Based Foraging environment on a 5×5 grid with two players and up to two food items. The setting is fully cooperative, meaning that each food item requires both players to load it jointly.

Agent 0's history-dependent policy is shown as below.

```python
from typing import List, Dict, Any, TypedDict, Union, Optional
import math, random

class GridWorldHistory(TypedDict):
    state: List[List[Union[int, float]]]
    action: List[List[Optional[int]]]

def history_dependent_policy_0(game_history: GridWorldHistory) -> List[float]:
    # Enhanced policy: probabilistic loading, softmax move selection, opponent intent, minimal exploration
    step = len(game_history['action'])
    max_steps = 50
    last_state = game_history['state'][-1]
    parsed = parse_grid_state(last_state)
    foods = [f for f in parsed['foods'] if f.get('alive', True)]
    agents = parsed['agents']
    me, other = agents[0], agents[1]
    grid_h = grid_w = 5

    # No foods => no-op
    if not foods:
        return [1.0, 0.0, 0.0, 0.0, 0.0, 0.0]
```


1 from typing import List , Dict , Any , Callable , TypedDict , Union , Optional 2 3 class 矩阵博弈历史 (TypedDict): 4 状态 : List [int]5 动作 : List [List [int]]6 7 def 历史_依赖_策略_0(博弈_历史 : 矩阵博弈历史) -> List[float]: 8 # 对手始终以1.0的概率采取动作0（承诺合作）。 9 # 最优响应是同样采取动作 0，以确保获得 11 点收益。 10 # 我正以合作回应你的合作。请继续采取动作0。 11 return [1.0 , 0.0 , 0.0]

智能体1的最终策略如下所示。

```
from typing import List, TypedDict

class 矩阵博弈历史 ( TypedDict ):
    状态 : 列表 [int]
    动作 : 列表 [列表 [整数 ]]

定义 历史_依赖_策略_1( 博弈_历史 : 矩阵博弈历史 ) -> 列表[浮点数 ]:
    # 对手 承诺 to 采取 动作 0 with 概率 1.0.
    # 最佳 回应 以确保获得 这11分的 收益的做法是也 play 动作 0.
    # I 正在承诺 以实现合作。 请匹配 me.
    返回 [1.0, 0.0, 0.0]
```

C.2 为合作觅食任务生成的代码策略

这里展示了任务 Foraging-5x5-2p-2f-coop-v3 单次运行的结果。该任务对应一个基于等级的觅食环境，其中有一个 5×5 网格、两名参与者和最多两个食物项。该设置是完全合作的，这意味着每个食物项都需要两名参与者共同装载。

智能体0的历史依赖策略如下所示。

1 from typing import List , Dict , Any , TypedDict , Union , Optional 2 import math , random 3 4 class GridWorldHistory (TypedDict): 5 state : List[List[Union [int , float]]] 6 action : List [List [Optional [int]]]7 8 def history_dependent_policy_0(game_history : GridWorldHistory) -> List[float]: 9 # 增强策略：概率式装载、softmax移动选择、对手意图、最小探索 10 step =len (game_history [' action ']) 11 max_steps = 5012 last_state =game_history [' state '][-1]13 parsed =parse_grid_state (last_state) 14 foods = [f for f in parsed [' foods ']if f. get (' alive ', True)]15 agents =parsed [' agents ']16 me , other =agents [0], agents [1]17 grid_h = grid_w = 518 19 # 没有食物 =>空操作 20 if not foods : 21 return [1.0 , 0.0 , 0.0 , 0.0 , 0.0 , 0.0]

```python
    # Immediate joint-load if possible
    if can_joint_load_any_food_two_agents(me, other, foods).get('can', False):
        probs = [0.0]*6
        probs[5] = 1.0
        return probs

    # Track opponent intent (visit counts)
    window = 5
    opp_visit = {i: 0 for i in range(len(foods))}
    start = max(0, step - window)
    for t in range(start, step):
        opp_act = game_history['action'][t][1]
        if opp_act in (1,2,3,4):
            prev = parse_grid_state(game_history['state'][t])['agents'][1]
            nxt = parse_grid_state(game_history['state'][t+1])['agents'][1]
            for i,f in enumerate(foods):
                d0 = abs(prev['x']-f['x']) + abs(prev['y']-f['y'])
                d1 = abs(nxt['x']-f['x']) + abs(nxt['y']-f['y'])
                if d1 < d0:
                    opp_visit[i] += 1

    # Score foods with share, distance, level, and opp_visit; then softmax-sample target
    scores = []
    total_level = me['level'] + other['level']
    for i,f in enumerate(foods):
        if total_level < f['level']:
            scores.append(-1e9)
        else:
            d = abs(me['x']-f['x']) + abs(me['y']-f['y'])
            share = me['level'] / total_level
            level_bonus = 1.0 / f['level']
            visit_bonus = opp_visit.get(i, 0) * 0.1
            scores.append(share/(d+1) + level_bonus + visit_bonus)
    T_food = max(0.1, (1.0 - step/max_steps))
    exp_s = [math.exp(s/T_food) for s in scores]
    sum_s = sum(exp_s) or 1.0
    weights = [e/sum_s for e in exp_s]
    r = random.random()
    cum = 0.0
    for idx,w in enumerate(weights):
        cum += w
        if r <= cum:
            target = foods[idx]
            break
    else:
        target = foods[weights.index(max(weights))]

    # If adjacent: probabilistic loading
    if is_adjacent((me['x'],me['y']), (target['x'],target['y'])):
        # partner-adjacent => always load
        if is_adjacent((other['x'],other['y']), (target['x'],target['y'])):
            probs = [0.0]*6
            probs[5] = 1.0
            return probs
        # time-aware load probability
        share = me['level'] / total_level
        load_p = min(1.0, share + (step/max_steps)*0.5)
        if me['level'] >= target['level']:
            load_p = min(1.0, load_p + 0.1)
```



```python
    # 如果可行，立即联合装载
    如果 能_联合_装载_任意_食物_两个_智能体（我，其他智能体，食物）.获取（’能’，False）：

        probs = [0.0]*6 probs[5] = 1.0 return probs # 跟踪对手意图（访问计数）
    window = 5opp_visit ={i: 0 for i in range ( len ( foods ))} start =max (0 , step - window ) for t in range ( start , step ): opp_act =game_history [’ action ’][t ][1] if opp_act in (1 ,2 ,3 ,4) : prev =parse_grid_state ( game_history [’ state ’][t ]) [’ agents ’][1]nxt =parse_grid_state ( game_history [’ state ’][t +1]) [’ agents ’][1]for i ,f in enumerate ( foods ): d0 =abs ( prev [’ x ’]-f[’ x ’]) +abs ( prev [’ y ’]-f[’ y ’]) d1 =abs ( nxt [’ x ’]-f[’ x ’]) +abs ( nxt [’ y ’]-f[’ y ’]) if d1 < d0 : opp_visit[i] += 1# 根据份额、距离、等级和 opp_visit 为食物评分；然后通过软最大值函数采样目标 scores = []total_level =me [’ level ’] +other [’ level ’] for i ,f in enumerate ( foods ): if total_level <f[’ level ’]: scores . append ( -1 e9 ) else : d =abs ( me [’ x ’]-f[’ x ’]) +abs ( me [’ y ’]-f[’ y ’]) share =me [’ level ’]/ total_level level_bonus = 1.0 / f[’ level ’]visit_bonus =opp_visit . get (i , 0) * 0.1 scores . append ( share /( d +1) +level_bonus +visit_bonus ) T_food = max (0.1 , (1.0 - step / max_steps )) exp_s = [math . exp (s/ T_food ) for s in scores ] sum_s =sum ( exp_s ) or 1.0 weights = [e/ sum_s for e in exp_s ]r =random . random () cum = 0.0 for idx ,w in enumerate ( weights ): cum +=w if r <= cum : target =foods [idx ]break else : target =foods [weights . index ( max ( weights ))]# 如果相邻：概率式装载 if is_adjacent (( me [’ x ’ ],me [’ y ’ ]) , ( target [’ x ’ ], target [’ y ’ ]) ): # 伙伴相邻 => 始终装载 if is_adjacent (( other [’ x ’ ], other [’ y ’ ]) , ( target [’ x ’ ], target [’ y ’ ]) ): probs = [0.0]*6 probs[5] = 1.0 return probs # 考虑时间的装载概率 share =me [’ level ’]/ total_level load_p =min (1.0 , share +( step / max_steps ) *0.5) if me [’ level ’] >=target [’ level ’]: load_p = min (1.0 , load_p + 0.1)
```

```
        # build movement vs load distribution
        valid_moves = get_valid_moves(me, foods, other, grid_h, grid_w, include_load=False)
        move_p = (1.0 - load_p) / max(len(valid_moves),1)
        probs = [0.0]*6
        for a in valid_moves:
            if a != 5:
                probs[a] = move_p
        probs[5] = load_p
        tot = sum(probs) or 1.0
        return [p/tot for p in probs]

    # Movement: softmax over neighbor moves toward target + small exploration
    occ = build_occupied_positions(foods, agents, exclude_agent_idx=0)
    neighbor_actions = []
    for a, (dx,dy) in {1:(-1,0), 2:(1,0), 3:(0,-1), 4:(0,1)}.items():
        nx, ny = me['x']+dx, me['y']+dy
        if 0 <= nx < grid_h and 0 <= ny < grid_w and (nx,ny) not in occ:
            neighbor_actions.append((a, (nx,ny)))
    if not neighbor_actions:
        return [1.0, 0.0, 0.0, 0.0, 0.0, 0.0]
    T_move = max(0.1, (1.0 - step/max_steps)*0.5)
    nb_scores = [ - (abs(pos[0]-target['x']) + abs(pos[1]-target['y'])) for _,pos in neighbor_actions ]
    exp_nb = [math.exp(s/T_move) for s in nb_scores]
    sum_nb = sum(exp_nb) or 1.0
    nb_probs = [e/sum_nb for e in exp_nb]
    # epsilon-greedy exploration
    epsilon = 0.05
    probs = [0.0]*6
    for (a,_), p_nb in zip(neighbor_actions, nb_probs):
        probs[a] = (1-epsilon)*p_nb + epsilon/len(neighbor_actions)
    return probs
```

Agent 1's history-dependent policy is shown as below.

```
from typing import List, Dict, Any, Callable, TypedDict, Union, Optional
import math, random

class GridWorldHistory(TypedDict):
    state: List[List[Union[int, float]]]
    action: List[List[Optional[int]]]

def history_dependent_policy_1(game_history: GridWorldHistory) -> List[float]:
    # Improved policy with softmax targeting, partner tracking, time-aware loading, and randomness to avoid deadlocks.
    step = len(game_history['action'])
    max_steps = 50
    last_state = game_history['state'][-1]
    parsed = parse_grid_state(last_state)
    foods = [f for f in parsed['foods'] if f.get('alive', True)]
    agents = parsed['agents']
    other, me = agents[0], agents[1]
    grid_h = grid_w = 5

    # no foods -> no-op
    if not foods:
        return [1.0,0.0,0.0,0.0,0.0,0.0]

    # seed randomness once
    if step == 0:
        random.seed(hash(tuple(last_state)))

```



```
82 # 构建移动与装载分布 83 valid_moves =get_valid_moves (me , foods , other , grid_h ,
gr
pr
[5
对
fo
1 ,
98
ac
0.(
ab
ac
ex
=
pr
```

智能体1的历史依赖策略如下所示。

```
1 from typing import List , Dict , Any , Callable , TypedDict , Union , Optional 2 import math , random 3 4 class GridWorldHistory ( TypedDict ): 5 state : List [ List [ Union [int , float ]]]6 action : List [ List [ Optional [int ]]]7 8 def history_dependent_policy_1 ( game_history : GridWorldHistory ) ->List [float ]: 9 # 改进的策略，采用软最大目标选择、伙伴跟踪、时间感知装载以及随机性来避免死锁。 10 step =len ( game_history [' action ' ]) 11 max_steps = 50 12 last_state =game_history [' state ' ][-1]13 parsed = parse_grid_state ( last_state ) 14 foods = [f for f in parsed [' foods ' ]if f. get (' alive ', True )]15 agents =parsed [' agents ' ]16 other , me =agents [0] , agents [1]17 grid_h = grid_w = 5 18 19 # 没有食物 ->空操作 20 if not foods : 21 return [1.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0]22 23 # 仅设置一次随机种子 24 if step == 0: 25 random . seed ( hash ( tuple ( last_state ))) 26
```

```python
    # immediate joint-load if possible
    if can_joint_load_any_food_two_agents(other, me, foods).get('can',
    False):
        return [0.0,0.0,0.0,0.0,0.0,1.0]

    # track opponent moves towards foods
    window = 5
    opp_visit = {i:0 for i in range(len(foods))}
    start = max(0, step - window)
    for t in range(start, step):
        opp_act = game_history['action'][t][0]
        if opp_act in (1,2,3,4):
            prev = parse_grid_state(game_history['state'][t])['agents'
    ][0]
            nxt = parse_grid_state(game_history['state'][t+1])['agents'
    ][0]
            for i,f in enumerate(foods):
                d0 = abs(prev['x']-f['x'])+abs(prev['y']-f['y'])
                d1 = abs(nxt['x']-f['x'])+abs(nxt['y']-f['y'])
                if d1 < d0:
                    opp_visit[i] += 1

    # scoring foods with softmax
    T_food = max(0.05, (1.0 - step/max_steps)**2)
    scores = []
    for i,f in enumerate(foods):
        if me['level']+other['level'] < f['level']:
            scores.append(-math.inf)
        else:
            d = abs(me['x']-f['x'])+abs(me['y']-f['y'])
            share = me['level']/(me['level']+other['level'])
            level_bonus = 1.0/f['level']
            visit_bonus = opp_visit.get(i,0)*0.1
            score = share/(d+1) + level_bonus + visit_bonus
            scores.append(score)
    exp_s = [math.exp(s/T_food) if s>-math.inf else 0.0 for s in scores]
    total_s = sum(exp_s) or 1.0
    weights = [e/total_s for e in exp_s]

    # choose target by probability distribution
    r = random.random()
    cum = 0.0
    for idx,w in enumerate(weights):
        cum += w
        if r <= cum:
            target = foods[idx]
            break
    else:
        target = foods[weights.index(max(weights))]

    # if adjacent: time-aware load probability
    if is_adjacent((me['x'],me['y']), (target['x'],target['y'])):
        share = me['level']/(me['level']+other['level'])
        load_p = min(1.0, share + (step/max_steps)*0.5)
        if me['level']>=target['level']:
            load_p = min(1.0, load_p+0.1)
        if step/max_steps < 0.3:
            load_p *= (step/max_steps)/0.3
        valid = get_valid_moves(me, foods, other, grid_h, grid_w,
    include_load=False)
        move_p = (1.0-load_p)/max(len(valid),1)
        probs = [0.0]*6
        for a in valid:
            probs[a] = move_p
        probs[5] = load_p
```



```
# 如果可以，立即联合装载 if can_joint_load_any_food_two_agents ( other , me , f
oods ). get (’ can ’, False ): return [0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,1.0]# 跟踪对手朝食物方向
的移动 window = 5opp_visit ={i:0 for i in range ( len ( foods ))} start =max (0 ,
step - window ) for t in range ( start , step ): opp_act =game_history [’ action ’][
t ][0]if opp_act in (1 ,2 ,3 ,4) : prev =parse_grid_state ( game_history [’ state ’][
t ]) [’ agents ’][0]nxt =parse_grid_state ( game_history [’ state ’][t +1]) [’
agents ’][0]for i ,f in enumerate ( foods ): d0 =abs ( prev [’ x ’]-f[’ x ’]) +abs
( prev [’ y ’]-f[’ y ’]) d1 =abs ( nxt [’ x ’]-f[’ x ’]) +abs ( nxt [’ y ’]-f[’
y ’]) if d1 < d0 : opp_visit[i] += 1# 使用软最大值函数对食物评分 T_food =max
(0.05 , (1.0 - step / max_steps ) **2) scores = []for i ,f in enumerate ( foods ): if
me [’ level ’]+other [’ level ’] <f[’ level ’]: scores . append (- math . inf )
else : d =abs ( me [’ x ’]-f[’ x ’]) +abs ( me [’ y ’]-f[’ y ’]) share =me [’
level ’]/( me [’ level ’]+other [’ level ’]) level_bonus = 1.0/ f[’ level ’]
visit_bonus =opp_visit . get (i ,0) *0.1 score =share /( d +1) +level_bonus +visit_
bonus scores . append ( score ) exp_s = [math . exp (s/ T_food ) if s>- math . inf
else 0.0 for s in scores ]total_s =sum ( exp_s ) or 1.0 weights = [e/ total_s for e i
n exp_s ]# 根据概率分布选择目标 r =random . random () cum = 0.0 for idx ,w in
enumerate ( weights ): cum +=w if r <= cum : target =foods [idx ]break else :
target =foods [ weights . index ( max ( weights ))]# 如果相邻：采用时间感知装载概率
if is_adjacent (( me [’ x ’],me [’ y ’]) , ( target [’ x ’], target [’ y ’])):
share =me [’ level ’]/( me [’ level ’]+other [’ level ’]) load_p =min (1.0 ,
share +( step / max_steps ) *0.5) if me [’ level ’]>=target [’ level ’]: load_p =
min (1.0 , load_p +0.1) if step / max_steps < 0.3: load_p *=( step / max_steps ) /0.3
valid =get_valid_moves (me , foods , other , grid_h , grid_w , include_load =False )
move_p =(1.0 - load_p )/ max ( len ( valid ) ,1) probs = [0.0]*6 for a in valid :
probs[a] = move_p probs[5] = load_p
```

```
        tot = sum(probs) or 1.0
        return [p/tot for p in probs]

    # movement: softmax over neighbor cells of target
    occ = build_occupied_positions(foods, agents, exclude_agent_idx=1)
    neighbor_cells = [(target['x']+dx, target['y']+dy)
                      for dx,dy in [(-1,0),(1,0),(0,-1),(0,1)]
                      if 0<=target['x']+dx<grid_h and 0<=target['y']+dy<
grid_w]
    valid_nb = [nb for nb in neighbor_cells if nb not in occ] or
neighbor_cells
    T_move = max(0.1, (1.0-step/max_steps)*0.5)
    nb_scores = []
    for nb in valid_nb:
        d = abs(me['x']-nb[0])+abs(me['y']-nb[1])
        nb_scores.append(-d)
    exp_nb = [math.exp(s/T_move) for s in nb_scores]
    total_nb = sum(exp_nb) or 1.0
    nb_probs = [e/total_nb for e in exp_nb]

    # map neighbor probs to action probs
    action_map = {
        1:(me['x']-1,me['y']), 2:(me['x']+1,me['y']),
        3:(me['x'],me['y']-1), 4:(me['x'],me['y']+1)
    }
    valid_moves = get_valid_moves(me, foods, other, grid_h, grid_w,
include_load=False)
    probs = [0.0]*6
    for nb,p_nb in zip(valid_nb,nb_probs):
        for a,pos in action_map.items():
            if a in valid_moves and pos==nb:
                probs[a] += p_nb

    # fallback: random valid move if stuck
    if sum(probs)==0:
        moves = [a for a in valid_moves if a!=5]
        if not moves:
            return [1.0,0.0,0.0,0.0,0.0,0.0]
        rnd = random.choice(moves)
        probs[rnd] = 1.0
        return probs

    tot = sum(probs)
    return [p/tot for p in probs]
```


88 tot =sum (probs) or 1.0 89 return [p/ tot for p in probs]90 91 # 移动：对目标的相邻单
id:
1 ,
gr
T_
10
10
or
10
10
m
pr
m
策
m
ch
re

D GLOSSARY

Table 1: Comprehensive Glossary of Symbols

Symbol	Meaning	Note
Markov Games & Environment		
$\mathcal{G}$	Markov Game tuple	$\langle N, \mathbb{S}, \{\mathbb{A}^i\}, \mathcal{T}, \{r^i\}, \gamma \rangle$
N	Set of agents	Index $i \in N$
$\mathbb{S}$	Global state space	$s \in \mathbb{S}$
$\mathbb{A}^i$	Action space for agent i	$\mathbb{A} = \times \mathbb{A}^i$ (Joint space)
$\mathcal{T}$	State transition function	$\mathcal{T} : \mathbb{S} \times \mathbb{A} \to \Delta(\mathbb{S})$
r^i	Reward function	$r^i : \mathbb{S} \times \mathbb{A} \to \mathbb{R}$
γ	Discount factor	$\gamma \in [0, 1)$
s_t, a_t^i	State and Action at timestep t	$\boldsymbol{a}_t$ denotes joint action
h_t	Interaction history	Sequence $(s_0, \boldsymbol{a}_0, \ldots, s_t)$
$\mathbb{H}$	Space of all histories	
$\mathcal{G}^i$	Perspective environment	Agent i's subjective view
J^i	Expected discounted return	Cumulative reward objective
J_{sw}	Social Welfare	Sum of all agents' returns
Policies & Representations		
π^i	Policy (Semantics)	Executable function $\pi^i : \mathbb{H} \to \Delta(\mathbb{A}^i)$
$\ulcorner \pi^i \urcorner$	Policy (Syntax)	Source code string / Gödel number
$\boldsymbol{\pi}$	Joint policy	$(\pi^i, \boldsymbol{\pi}^{-i})$
Π	General policy space	
Π_{prog}	Space of programmatic policies	Set of valid programs in the language
θ^i	Neural network parameters	Used in standard DRL
$\Delta(\mathbb{X})$	Probability simplex over $\mathbb{X}$	
m	Opponent modeling function	Decoder m : Signal $\to \Pi$
ϕ	Policy encoder	$\phi : \Pi \to$ Signal
Method (PIBR) & Optimization		
φ^i	Best-Response Operator	Meta-function $\{0,1\}^* \to \{0,1\}^*$
$\mathcal{L}^i$	Learning dynamics	Update rule $\theta_{k+1} = \mathcal{L}(\theta_k)$
K	Outer loop iterations	Number of mutual adaptation rounds
T	Inner loop iterations	Number of operator optimization steps
$\mathcal{L}_{\text{utility}}$	Utility Feedback Loss	Based on game trajectories
$\mathcal{L}_{\text{test}}$	Unit Test Loss	Based on runtime/syntax correctness
prompt	LLM Instruction	Input to operator φ
Logic & Game Theory (Appendix)		
π_{FB}	FairBot Program	Cooperates iff proved opponent cooperates
$\mathcal{T}$	Base Theory	Formal system (e.g., Peano Arithmetic)
$\vdash$	Provability relation	$\mathcal{T} \vdash P$ means P is provable in $\mathcal{T}$
$\Box P$	Modal operator	"It is provable that P"
r_T, r_R	Temptation/Reward payoffs	Prisoner's Dilemma parameters
r_P, r_S	Punishment/Sucker payoffs	Prisoner's Dilemma parameters

表 1：完整符号释义表

符号	含义	Note
马尔可夫博弈与环境		
$\mathcal{G}$	马尔可夫博弈元组	$\langle N, \mathbb{S}, \{\mathbb{A}^i\}, \mathcal{T}, \{r^i\}, \gamma \rangle$
N	智能体集合	索引 $i \in N$
$\mathbb{S}$	全局状态空间	$s \in \mathbb{S}$
$\mathbb{A}^i$	智能体 i 的动作空间	$\mathbb{A} = \times \mathbb{A}^i$ （联合空间）
$\mathcal{T}$	状态转移函数	$\mathcal{T}$: $\mathbb{S} \times \mathbb{A} \to \Delta(\mathbb{S})$
r^i	奖励函数	r^i: $\mathbb{S} \times \mathbb{A} \to \mathbb{R}$
γ	折扣因子	$\gamma \in [0, 1)$
s_t, a_t^i	时间步 t 的状态和动作	$\boldsymbol{a}_t$ 表示联合动作
h_t	交互历史	序列 $(s_0, \boldsymbol{a}_0, \ldots, s_t)$
$\mathbb{H}$	所有历史的空间	
$\mathcal{G}^i$	视角环境	智能体 i 的主观视角
J^i	期望折扣回报	累积奖励目标
J_{sw}	社会福利	所有智能体的回报之和
策略与表示		
π^i	策略（语义）	可执行函数 π^i： $\mathbb{H} \to \Delta(\mathbb{A}^i)$
$\ulcorner \pi^i \urcorner$	策略（语法）	源代码字符串 / 哥德尔数
$\boldsymbol{\pi}$	联合策略	$(\pi^i, \boldsymbol{\pi}^{-i})$
Π	一般策略空间	
Π_{prog}	程序化策略空间	该语言中有效程序的集合
θ^i	神经网络参数	用于标准深度强化学习
$\Delta(\mathbb{X})$	$\mathbb{X}$ 上的概率单纯形	
m	对手建模函数	解码器 m： 信号 $\to \Pi$
ϕ	策略编码器	ϕ： $\Pi \to$ 信号
方法（*PIBR*）与优化		
φ^i	最优响应算子	元函数 $\{0,1\}^* \to \{0,1\}^*$
$\mathcal{L}^i$	学习动力学	更新规则 $\theta_{k+1} = \mathcal{L}(\theta_k)$
K	外循环迭代	相互适应轮次数
T	内循环迭代	算子优化步数
$\mathcal{L}_{\text{utility}}$	效用反馈损失	基于博弈轨迹
$\mathcal{L}_{\text{test}}$	单元测试损失	基于运行时与语法正确性
提示词	大语言模型指令	算子 φ 的输入
逻辑学与博弈论（附录）		
π_{FB}	FairBot 程序	当且仅当证明对手会合作时才合作
$\mathcal{T}$	基础理论	形式系统（例如，皮亚诺算术）
$\vdash$	可证明性关系	$\mathcal{T} \vdash P$ 表示 P 在 $\mathcal{T}$ 中可证明
$\Box P$	模态算子	"P 是可证明的"
r_T, r_R	诱惑收益／奖励收益	囚徒困境参数
r_P, r_S	惩罚收益／受骗者收益	囚徒困境参数